

전차환경을 이용한 AI프로젝트 설계

강사소개

김성식

(주)방배동밸리 대표이사

sskim@bvdev.co.kr

<주요 경력>

삼성 타이젠OS 개발 프로젝트(스트리밍 통신)

일본 SEASON 카드 기간계 개발 프로젝트

한국국방연구원 프로젝트

일본 NTT DATA 증권 선물 시스템

닐슨 Spaceman POG시스템 개발

LH 사이버모델하우스 및 항공 VR개발

일본 AGC 아시아거점 스마트팩토리 선행개발

목차

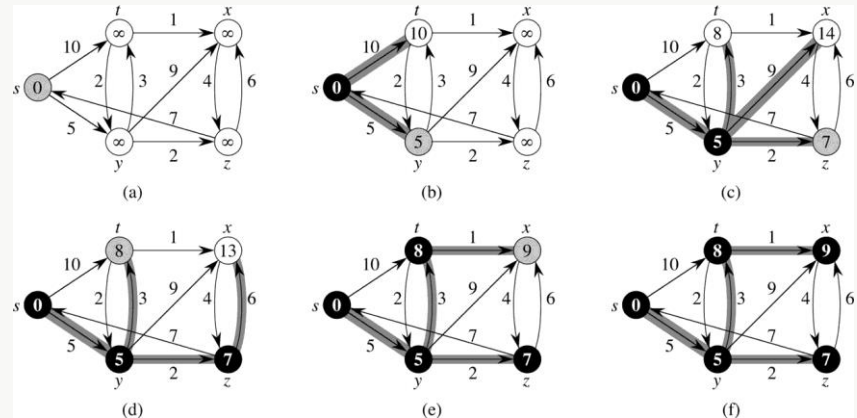
1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해
 - 1) 다익스트라 알고리즘 (Dijkstra Algorithm)
 - 2) 플로이드-워셜 알고리즘 (Floyd-Warshall Algorithm)
 - 3) 벨만-포드 알고리즘 (Bellman-Ford-Moore Algorithm)
 - 4) A* 알고리즘
 - 5) 다익스트라 vs A*
2. 강화학습 실습
 - 1) Frozen Lake를 이용한 강화학습 실습
 - 2) Cart pole을 이용한 강화학습 실습

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

1) 다익스트라 알고리즘 (Dijkstra Algorithm)

Dijkstra의 최단 경로 알고리즘은 네트워크에서 하나의 시작 정점으로부터 모든 다른 정점까지의 최단 경로를 찾는 알고리즘이다. 최단 경로는 경로의 길이순으로 정해진다. Dijkstra의 알고리즘에서는 시작 정점에서 집합 S에 있는 정점만을 거쳐서 다른 정점으로 가는 최단 거리를 기록하는 배열이 반드시 있어야 한다.

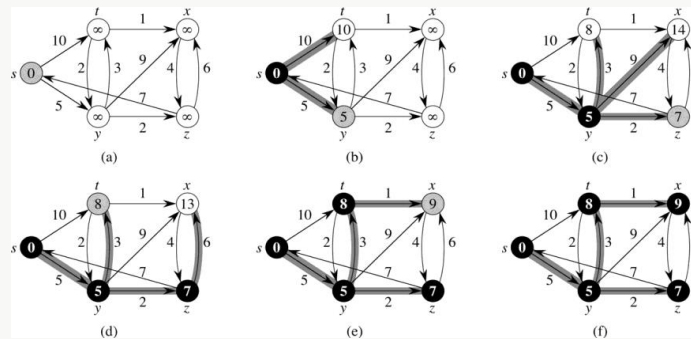
	0	1	2	3	4	5	6
0	0	7	∞	∞	3	10	∞
1	7	0	4	10	2	6	∞
2	∞	4	0	2	∞	∞	∞
3	∞	10	2	0	11	9	4
4	3	2	∞	11	0	∞	5
5	10	6	∞	9	∞	0	∞
6	∞	∞	∞	4	5	∞	0



1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

1) 다익스트라 알고리즘 (Dijkstra Algorithm)

다익스트라 알고리즘의 동작 과정

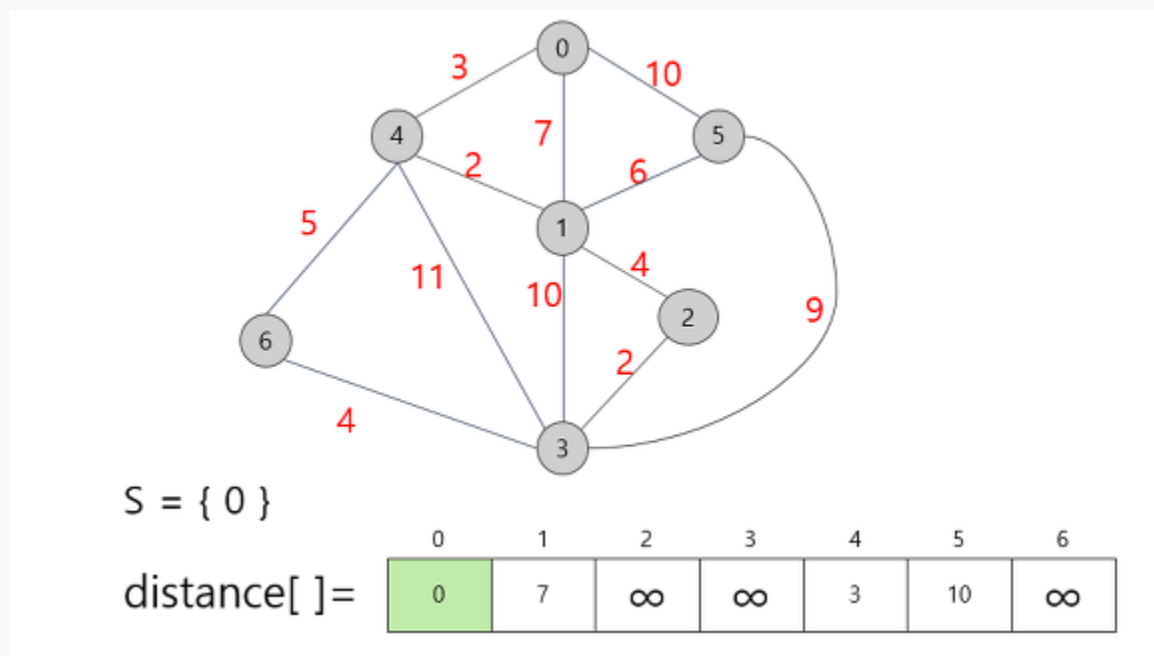


- ① 현재 선택한 정점(처음엔 시작점)에 곧장 연결되고, 아직 방문하지 않은 정점들을 모두 본다.
- ② 선택한 정점과 보고 있는 정점 사이의 거리와 시작 정점과 선택한 정점까지의 최단거리의 합이 현재까지 구한 시작 정점과 보고 있는 정점 사이의 거리보다 짧을 경우, 이를 갱신해준다.
- ③ 1, 2번 수행이 모두 끝난 이후, 아직 방문하지 않은 정점들 중 시작점과의 거리가 가장 짧은 정점을 선택한다.
- ④ 방문하지 않은 정점이 존재하여 정점을 선택했다면, 현재 구한 시작점으로부터의 현재 선택한 정점 사이의 거리는 최단거리이다.
- ⑤ 방문하지 않은 정점이 존재하지 않는다면, 수행을 종료한다. 그렇지 않으면 1번으로 돌아가 수행을 반복한다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

1) 다익스트라 알고리즘 (Dijkstra Algorithm)

다익스트라 알고리즘의 동작 과정

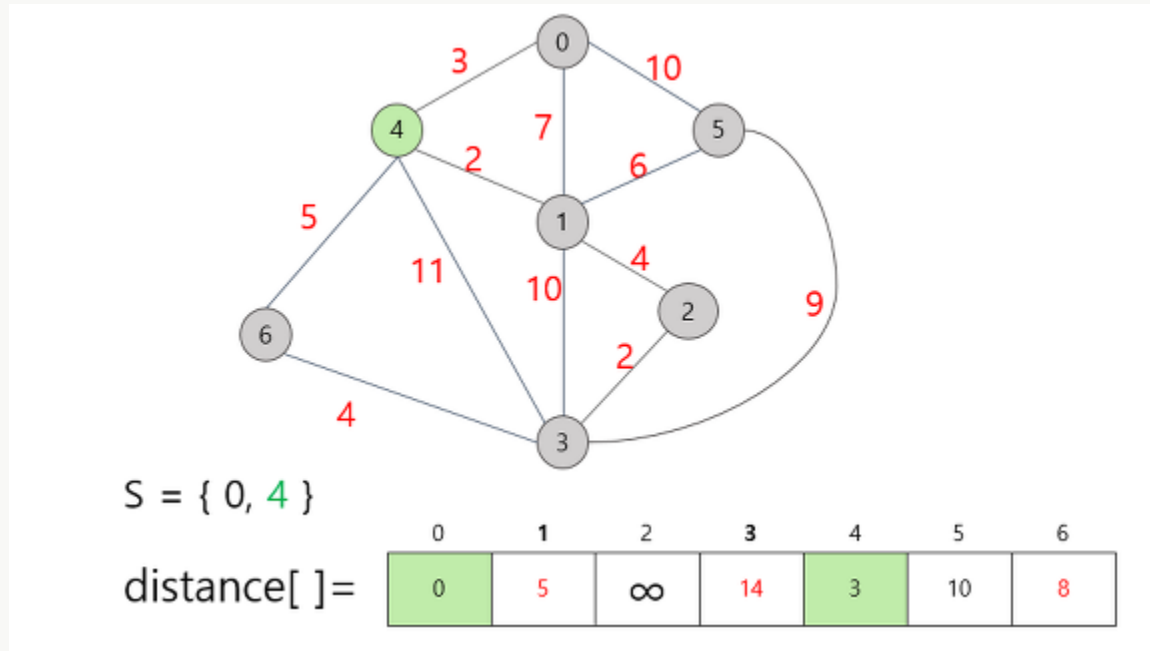


시작 정점을 0으로 잡고, 각 지점까지의 거리를 표시했다. 직접적으로 가는 경로가 없는 경우 무한대로 표시되어 있다. 표시 되어 있는 거리 중 가장 짧은 거리는 정점 4까지의 거리인 3이므로 정점 4를 집합 S에 추가시켜준다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

1) 다익스트라 알고리즘 (Dijkstra Algorithm)

다익스트라 알고리즘의 동작 과정



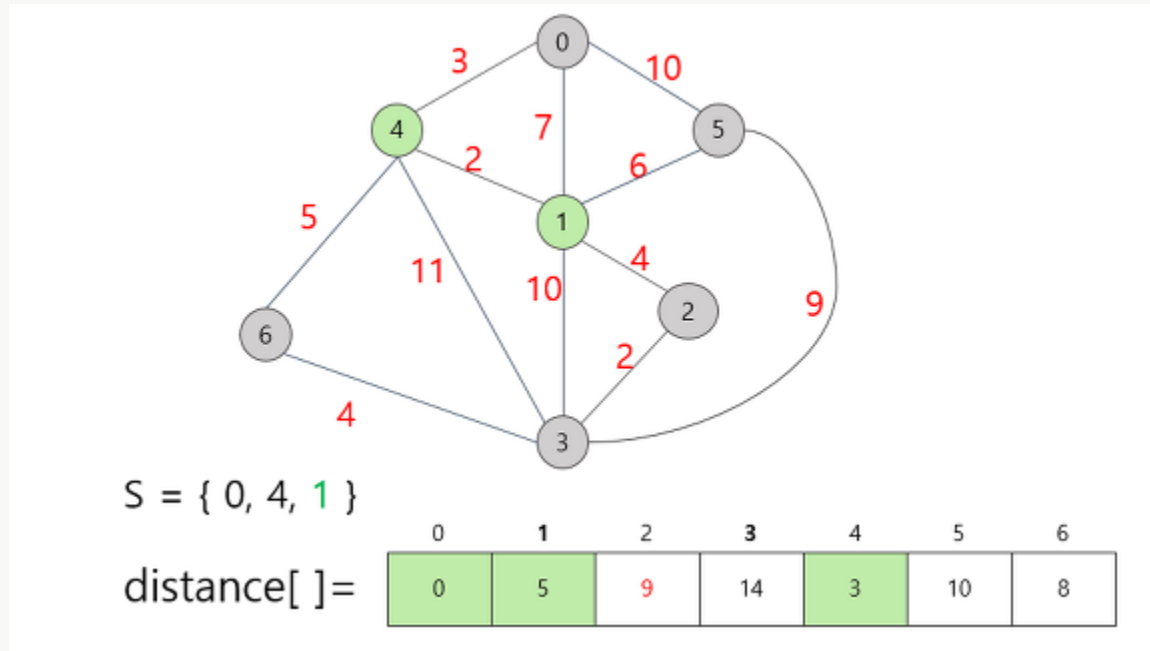
시작 정점을 0으로 잡고, 각 지점까지의 거리를 표시했다. 직접적으로 가는 경로가 없는 경우 무한대로 표시되어 있다. 표시 되어 있는 거리 중 가장 짧은 거리는 정점 4까지의 거리인 3이므로 정점 4를 집합 S에 추가시켜준다.

또한 새로운 정점 4를 통해 갈 때 더 짧은 경로가 발견 된다면 그 정보 또한 갱신을 해준다. 구체적으로는 위의 0번 정점만을 선택했을 때, 1번 정점까지의 거리가 7이었는데, 4번 정점을 택함으로써 이 거리가 5로 줄어들었다. 즉 더 짧은 거리가 나타나거나 기존의 무한대에서 직접 경로가 생겼을 때 거리(비용) 값을 갱신해준다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

1) 다익스트라 알고리즘 (Dijkstra Algorithm)

다익스트라 알고리즘의 동작 과정

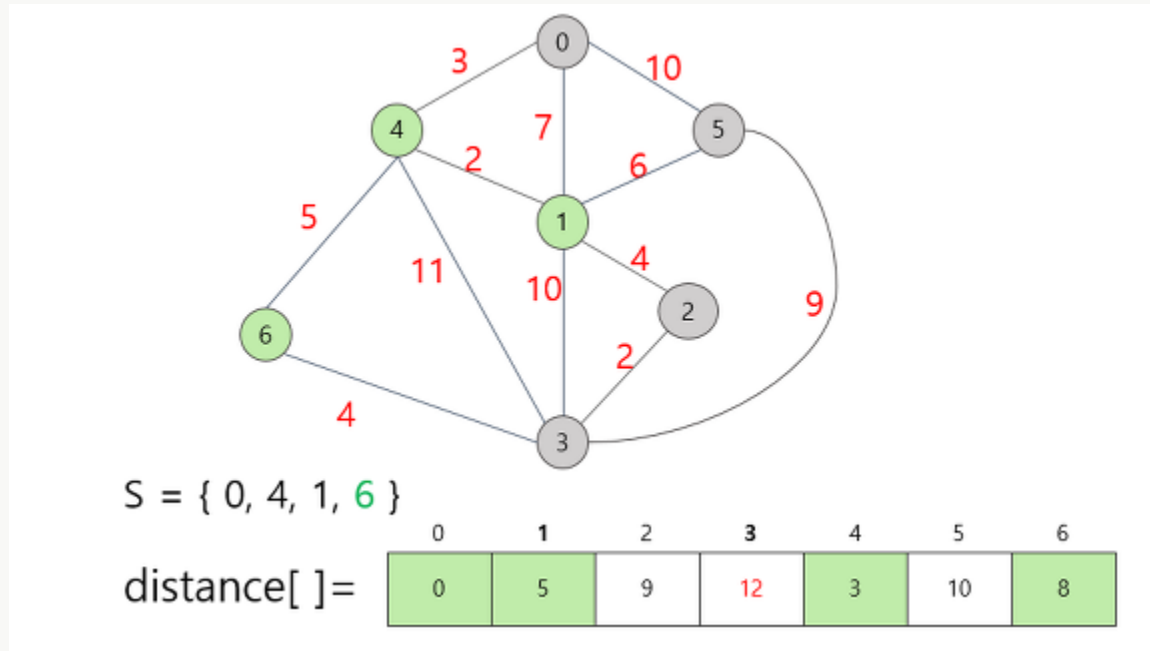


갱신된 정보는 2번 정점에 대한 가중치다. 1번 정점을 추가함으로써 2번 정점까지 직접적으로 갈 수 있게 되었으므로 무한대의 값에서 구체적인 가중치인 9로 수정해준다. 다음으로, 현재까지의 distance 배열값을 기준으로 가장 작은 값은 6번 정점의 8이므로, 6번 정점을 택한다. 마찬가지로 갱신할 수 있는 정보는 갱신해준다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

1) 다익스트라 알고리즘 (Dijkstra Algorithm)

다익스트라 알고리즘의 동작 과정

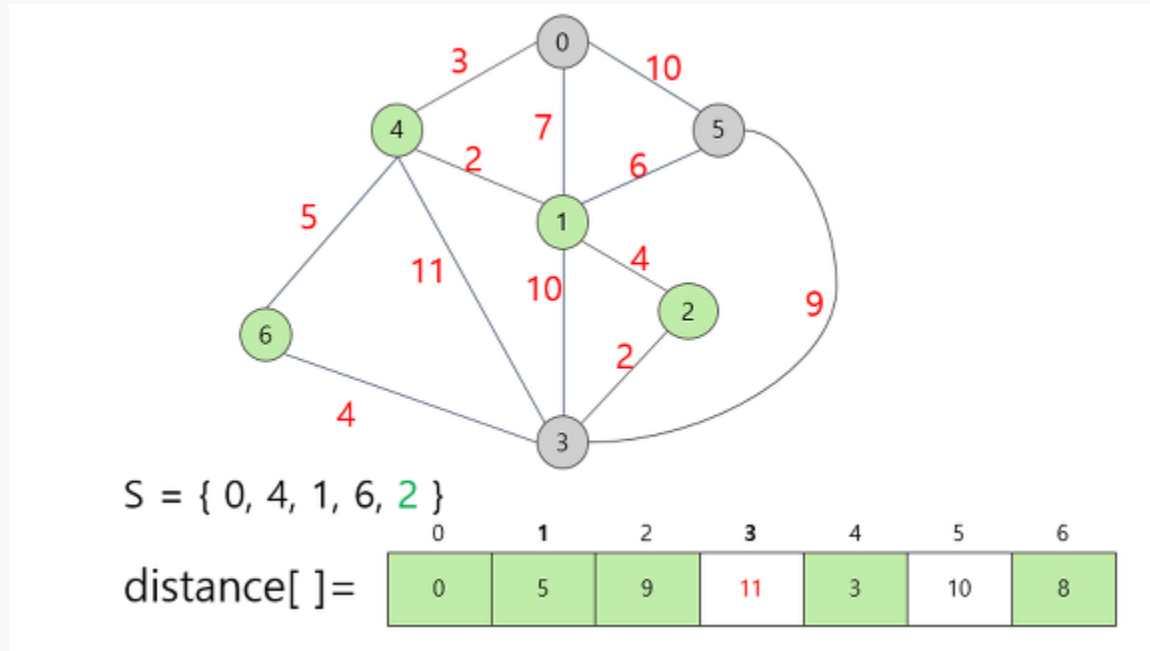


6번 정점을 집합 S에 추가함으로써 갱신할 수 있는 정보는 정점 3까지의 거리다. 기존의 14에서 12로 거리가 단축되었으므로, 이 값을 갱신했다. 이제 다음에 택할 정점을 생각해보자. distance 배열에서 가장 작은 가중치는 9이므로 정점 2를 택한다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

1) 다익스트라 알고리즘 (Dijkstra Algorithm)

다익스트라 알고리즘의 동작 과정



정점 2를 집합 S에 추가함으로써 정점 3까지의 거리가 갱신되었다. 남은 두 개의 과정은 그림을 올리지 않아도 유추가 가능하다. 가중치가 가장 적은 5번 정점을 선택하고, 갱신할 정보가 있다면 갱신한다. 그 다음은 마지막 정점인 3번 정점을 택한다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

1) 다익스트라 알고리즘 (Dijkstra Algorithm)

다익스트라 알고리즘의 특징

- 그리디 알고리즘: 매 상황에서 방문하지 않은 가장 비용이 적은 노드 선택. 단계 거치며 한번 처리된 노드(이미 방문 처리)의 최단 거리는 고정되어 바뀌지 않음
- 한 단계당 하나의 노드에 대한 최단 거리를 확실히 찾는 것으로 이해
- 다익스트라 수행 후, 테이블에 각 노드까지의 최단 거리 정보가 저장됨. 단계마다 방문하지 않은 노드 중 최단 거리 가장 짧은 노드 선택하기 위해, 매 단계마다 1차원 테이블의 모든 원소를 확인(순차 탐색)→ 순차 탐색: 시간복잡도 $O(N^2)$
- $O(N)$ 번에 걸쳐서 최단거리가 가장 짧은 노드. 매번 선형 탐색해야 하고, 현재 노드와 연결된 노드 매번 확인. 전체 노드 개수가 5000이하라면 이렇게 풀 수 있으나 10,000개 넘어가면 해결 어려움.
- 힙 자료구조 이용하는 다익스트라 알고리즘의 시간복잡도는 $O(E \log V)$. 노드를 하나씩 꺼내 검사하는 반복문은 노드의 개수 V 이상의 횟수로 처리되지 않음. 결과적으로 현재 우선순위 큐에서 꺼낸 노드와 연결된 다른 노드들을 확인하는 총 횟수는 최대 간선의 개수(E)만큼 연산이 수행될 수 있음 (내부적으로 힙이 정렬해주기 때문). 직관적으로 전체 과정은 E 개의 원소를 우선순위 큐에 넣었다가 모두 빼내는 연산과 매우 유사함.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

1) 다익스트라 알고리즘 (Dijkstra Algorithm)

```
INF = int(1e9)
# 무한을 의미하는 값으로 10억을 설정
# 노드의 개수, 간선의 개수
n,m = map(int, input().split())
# 시작 노드 번호
start = int(input())
# 각 노드에 연결되어 있는 노드에 대한 정보 담은 리스트 만들기
graph = [ [ ] for i in range(n+1)]
# 최단 거리 테이블을 모두 무한으로 초기화
distance = [INF] * (n+1)
# 모든 간선 정보 입력 받기
for _ in range(m):
    a, b, c = map(int, input().split())
    # a번 노드에서 b번 노드로 가는 비용이 c라는 의미
    graph[a].append((b,c))
def dijkstra(start):
    q = [ ]
    # 시작 노드로 가기 위한 최단 경로는 0으로 설정하여, 큐에 삽입
    heapq.heappush(q, (0, start))
    distance[start] = 0
    while q:
        # 가장 최단 거리가 짧은 노드에 대한 정보 꺼내기
        dist, now = heapq.heappop(q)
        # 현재 노드가 이미 처리된 적이 있는 노드라면 무시
        if distance[now] < dist:
            continue
        # 현재 노드와 연결된 다른 인접한 노드들을 확인
        for i in graph[now]:
            cost = dist + i[1]
            # 현재 노드 거쳐서, 다른 노드로 이동하는 거리가 더 짧은 경우
            if cost < distance[i[0]]:
                distance[i[0]] = cost
                heapq.heappush(q, (cost, i[0]))
    dijkstra(start)
# 모든 노드로 가기 위한 최단 거리 출력
for i in range(1, n+1):
    # 도달할 수 없는 경우, 무한이라고 출력
    if distance[i] == INF:
        print("INFINITY")
    # 도달할 수 있는 경우 거리를 출력
    else:
        print(distance[i])
```

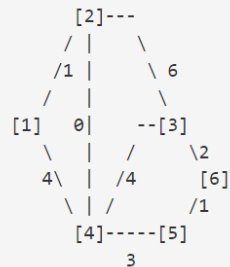
```
6 8
1
4 5 3
2 4 0
1 4 4
1 2 1
5 6 1
3 6 2
3 2 6
3 4 4
0
1
INFINITY
1
4
5
```

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

1) 다익스트라 알고리즘 (Dijkstra Algorithm)

A전차 대대 대대장은 상부로부터 기동명령을 하달 받았다.

작전에 앞서 첩보작전으로 작성된 지도가 있다. N ($1 \leq N \leq 50,000$) 개의 거점과, 전차가 이동가능한 길인 M ($1 \leq M \leq 50,000$) 개의 양방향 길이 그려져 있고, 각각의 길에는 C_i ($0 \leq C_i \leq 1,000$) 대의 적 전차가 매복되어 있다. 이동경로는 각 거점 A_i 와 B_i ($1 \leq A_i \leq N$; $1 \leq B_i \leq N$; $A_i \neq B_i$)를 잇는다. 두 개의 거점은 하나 이상의 길로 연결되어 있을 수도 있다. A전차 대대는 거점1에 있고 작전목표지점은 마지막 거점번호 N 에 있다.
(지도를 참고)



대대장이 선택할 수 있는 최선의 통로는 1 -> 2 -> 4 -> 5 -> 6 이다. 왜냐하면 최소한의 적을 만나는 경로로 이동했을때 마주치는 적은 $1 + 0 + 3 + 1 = 5$ 이기 때문이다. 지도가 주어지고, 지나가는 길에 매복중인 적 전차의 수가 주어질 때 적 전차를 마주치는 최소값은 얼마일까? (가는 길의 길이는 고려하지 않는다.)

입력

첫째 줄에 N 과 M 이 공백을 사이에 두고 입력
둘째 줄부터 $M+1$ 번째 줄까지 세 개의 정수 A_i, B_i, C_i
입력

```
6 8
4 5 3
2 4 0
4 1 4
2 1 1
5 6 1
3 6 1
3 2 6
3 4 4
```

출력

첫째 줄에 이동간 마주치는적 전차의 최소값을 출력한다.

```
5
```

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

1) 다익스트라 알고리즘 (Dijkstra Algorithm)

```
import heapq
#import sys
#input = sys.stdin.readline
INF = int(1e9)
# 무한을 의미하는 값으로 10억을 설정
# 노드의 개수, 간선의 개수
n,m = map(int, input().split())
# 각 노드에 연결되어 있는 노드에 대한 정보 담는 리스트 만들기
graph = [ [ ] for i in range(n+1)]
# 최단 거리 테이블을 모두 무한으로 초기화
distance = [INF] * (n+1)
```

```
for _ in range(m):
    a, b, c = map(int, input().split())
    graph[a].append((b, c))
    graph[b].append((a, c))
```

```
def dijkstra(start):
    q = []
    heapq.heappush(q, (0, start))
    distance[start] = 0
    while q:
        dist, now = heapq.heappop(q)
        if distance[now] < dist:
            continue
        for i in graph[now]:
            cost = dist + i[1]
            if cost < distance[i[0]]:
                distance[i[0]] = cost
                heapq.heappush(q, (cost, i[0]))
```

```
dijkstra(1)
```

```
print(distance[-1])
```

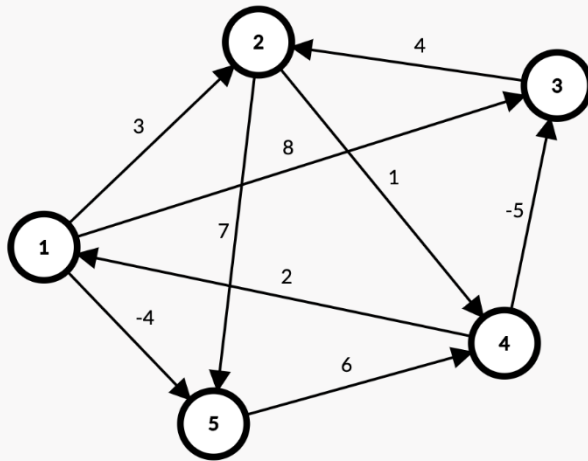
1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

2) 플로이드-워셜 알고리즘 (Floyd-Warshall Algorithm)

모든 노드에서 다른 모든 노드까지의 최단 경로.

매 단계마다 방문하지 않은 노드 중 최단거리 찾는 과정 필요x(우선순위큐 안써도됨.)
2차원 테이블에 최단거리 정보 저장.

각 단계마다 특정 노드 K 거쳐가는 경우 확인. $\min(A \rightarrow B, A \rightarrow K \rightarrow B)$

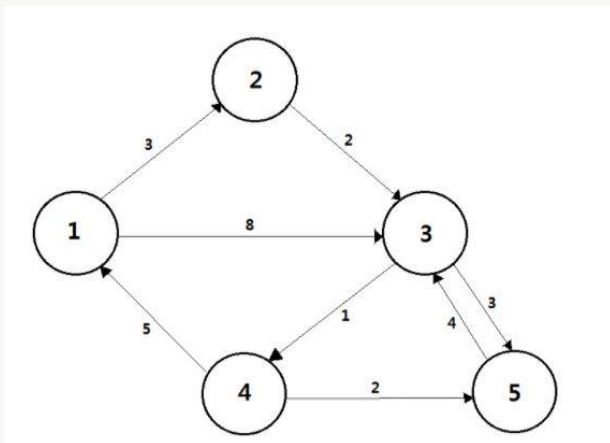


$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0, \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & \text{if } k \geq 1. \end{cases}$$

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

2) 플로이드-워셜 알고리즘 (Floyd-Warshall Algorithm)

플로이드-워셜 알고리즘의 동작 과정



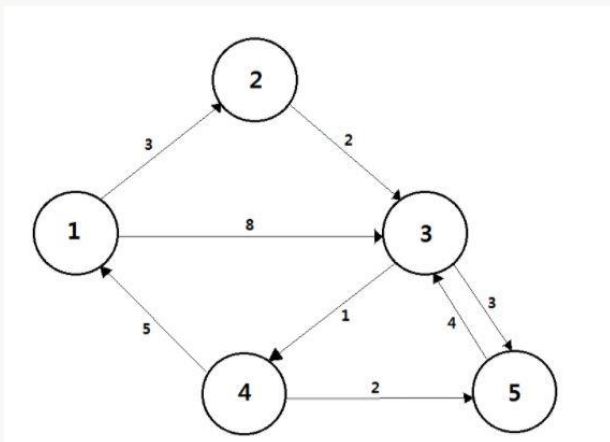
	1	2	3	4	5
1	0	3	8	INF	INF
2	INF	0	2	INF	INF
3	INF	INF	0	1	3
4	5	INF	INF	0	2
5	INF	INF	4	INF	0

가중치를 가지는 방향 그래프가 위 그림처럼 있다고 하자. 모든 정점의 최단 경로를 구해야 하기 때문에 2차원 배열로 표현한다. $D[i][j]$ 는 i 번 정점에서 j 번 정점으로 가는 최소 비용이다. 자신에게 가는 비용은 0, 1개의 간선을 거쳐서 갈 수 있는 경로는 가중치, 나머지는 INF로 초기 상태를 설정한다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

2) 플로이드-워셜 알고리즘 (Floyd-Warshall Algorithm)

플로이드-워셜 알고리즘의 동작 과정



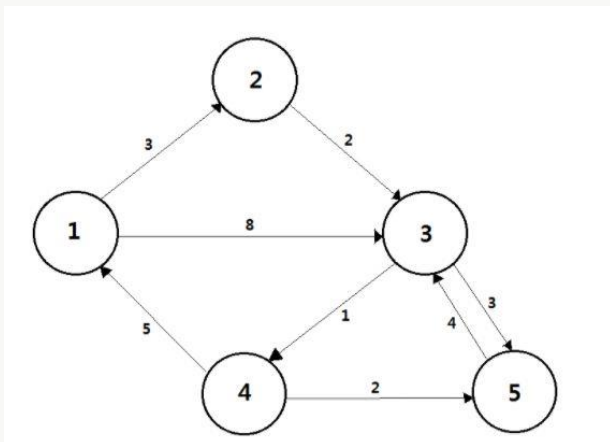
	1	2	3	4	5
1	0	3	8	INF	INF
2	INF	0	2	INF	INF
3	INF	INF	0	1	3
4	5	8	13	0	2
5	INF	INF	4	INF	0

1번 정점을 중간 경로인 기준 정점으로 정하고 모든 정점에서 1번 정점을 거쳐 모든 정점으로 가는 비용을 갱신해 준다. 4-2로 가는 비용 INF보다 4-1-2로 가는 비용 8이 저렴하기 때문에 갱신된다. 4-3으로 가는 비용 INF보다 4-1-3으로 가는 비용 13이 저렴하기 때문에 갱신된다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

2) 플로이드-워셜 알고리즘 (Floyd-Warshall Algorithm)

플로이드-워셜 알고리즘의 동작 과정



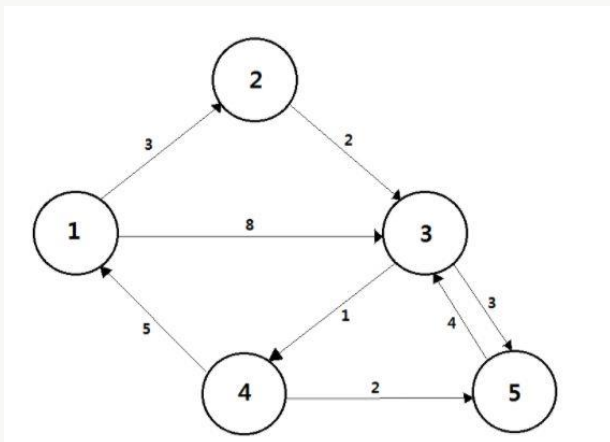
	1	2	3	4	5
1	0	3	5	INF	INF
2	INF	0	2	INF	INF
3	INF	INF	0	1	3
4	5	8	10	0	2
5	INF	INF	4	INF	0

2번 정점을 중간 경로인 기준 정점으로 정하고 모든 정점에서 2번 정점을 거쳐 모든 정점으로 가는 비용을 갱신해 준다. 1-3으로 가는 비용 8보다 1-2-3으로 가는 비용 5가 저렴하기 때문에 갱신된다. 4-3으로 가는 비용 13보다 4-2-3으로 가는 비용이 10으로 저렴하기 때문에 갱신된다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

2) 플로이드-워셜 알고리즘 (Floyd-Warshall Algorithm)

플로이드-워셜 알고리즘의 동작 과정



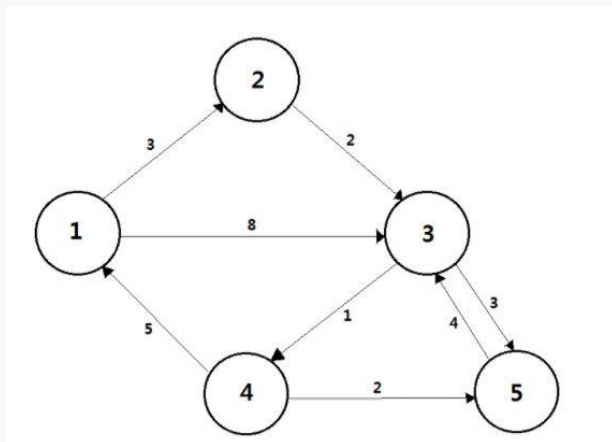
	1	2	3	4	5
1	0	3	5	6	8
2	INF	0	2	3	5
3	INF	INF	0	1	3
4	5	8	10	0	2
5	INF	INF	4	5	0

3번 정점을 중간 경로인 기준 정점으로 정하고 위와 같은 방법으로 모든 정점에 대해 갱신해 준다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

2) 플로이드-워셜 알고리즘 (Floyd-Warshall Algorithm)

플로이드-워셜 알고리즘의 동작 과정



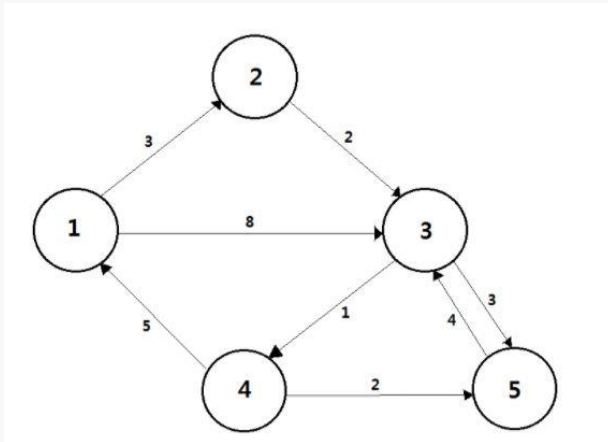
	1	2	3	4	5
1	0	3	5	6	8
2	8	0	2	3	5
3	6	9	0	1	3
4	5	8	10	0	2
5	10	13	4	5	0

4번 정점을 중간 경로인 기준 정점으로 정하고 위와 같은 방법으로 모든 정점에 대해 갱신해 준다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

2) 플로이드-워셜 알고리즘 (Floyd-Warshall Algorithm)

플로이드-워셜 알고리즘의 동작 과정



	1	2	3	4	5
1	0	3	5	6	8
2	8	0	2	3	5
3	6	9	0	1	3
4	5	8	6	0	2
5	10	13	4	5	0

5번 정점을 중간 경로인 기준 정점으로 정하고 위와 같은 방법으로 모든 정점에 대해 갱신해 준다. 정점의 수만큼 반복하여 최종적으로 모든 정점으로 가는 최소 비용을 구하였다.

모든 정점에 대해 모든 정점을 거쳐 모든 정점으로 가는 비용을 비교하기 때문에 플로이드 워셜 알고리즘은 $O(V^3)$ 의 시간 복잡도를 가진다. 플로이드 워셜 알고리즘은 시간 복잡도가 큰 대신 구현하기가 정말 간단한 장점이 있다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

2) 플로이드-워셜 알고리즘 (Floyd-Warshall Algorithm)

플로이드-워셜 알고리즘의 특징

다익스트라, 벨만-포드 알고리즘은 한 정점에서 다른 정점으로 가는 최단 경로를 구하는 알고리즘이었다면, 플로이드-워셜은 모든 정점에서 모든 정점으로의 최단 경로를 구하는 알고리즘이다

그래프에서 모든 정점 사이의 최단 거리를 구하기 위한 알고리즘

다익스트라 알고리즘을 모든 정점에서 수행한 것과 같은 알고리즘이지만 플로이드 와샬 알고리즘은 구현이 간단하다.

음수 가중치에 대한 처리가 어려운 다익스트라 알고리즘에 비해 플로이드 와샬 알고리즘은 사이클이 없는 경우 음수 가중치 처리가 가능하다

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

2) 플로이드-워셜 알고리즘 (Floyd-Warshall Algorithm)

```
INF = int(1e9) # 무한을 의미하는 10억 설정

# 노드의 개수 및 간선의 개수 입력 받기
n,m = map(int,input().split())
# 2차원 리스트를 만들고, 무한으로 초기화
graph = [[INF]*(n+1) for _ in range(n+1)]

# 자기 자신에서 자기 자신으로 가는 비용은 0으로 초기화
for a in range(1, n+1):
    for b in range(1, n+1):
        if a==b:
            graph[a][b] = 0
# 각 간선에 대한 정보 입력받아, 그 값으로 초기화
for _ in range(m):
    # a에서 b로 가는 비용은 c
    a, b, c = map(int, input().split())
    graph[a][b] = c

# 점화식에 따라 플로이드 워셜 수행
for k in range(1, n+1):
    for a in range(1, n+1):
        for b in range(1, n+1):
            graph[a][b] = min(graph[a][b], graph[a][k]+ graph[k][b])

# 수행된 결과 출력
for a in range(1, n+1):
    for b in range(1, n+1):
        # 도달할 수 없는 경우, 무한으로 출력
        if graph[a][b] == INF:
            print(a,"->",b,":",INFINITY", end= ' ')
        # 도달할 수 있는 경우 거리를 출력
        else:
            print(a,"->",b,":",graph[a][b], end=' ')
    print()
```

```
5 8
1 2 3
1 3 8
2 3 2
4 1 5
3 4 1
4 5 2
3 5 3
5 3 4
1 -> 1 : 0
1 -> 2 : 3
1 -> 3 : 5
1 -> 4 : 6
1 -> 5 : 8
2 -> 1 : 8
2 -> 2 : 0
2 -> 3 : 2
2 -> 4 : 3
2 -> 5 : 5
3 -> 1 : 6
3 -> 2 : 9
3 -> 3 : 0
3 -> 4 : 1
3 -> 5 : 3
4 -> 1 : 5
4 -> 2 : 8
4 -> 3 : 6
4 -> 4 : 0
4 -> 5 : 2
5 -> 1 : 10
5 -> 2 : 13
5 -> 3 : 4
5 -> 4 : 5
5 -> 5 : 0
```

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

2) 플로이드-워셜 알고리즘 (Floyd-Warshall Algorithm)

V개의 거점과 E개의 도로로 구성되어 있는 지역이 있다. 도로는 거점과 거점 사이에 놓여 있으며, 일방 통행 도로이다. 거점에는 편의상 1번부터 V번까지 번호가 매겨져 있다고 하자.

당신은 도로를 따라 기동훈련을 하기 위한 경로를 찾으려고 한다. 기동훈련을 한 후에는 다시 시작점으로 돌아오는 것이 좋기 때문에, 우리는 사이클을 찾기를 원한다. 단, 당신은 기동훈련을 매우 귀찮아하므로, 사이클을 이루는 도로의 길이의 합이 최소가 되도록 찾으려고 한다.

도로의 정보가 주어졌을 때, 도로의 길이의 합이 가장 작은 사이클을 찾는 프로그램을 작성하시오. 두 거점을 왕복하는 경우도 사이클에 포함됨에 주의한다.

입력

첫째 줄에 V와 E가 빈칸을 사이에 두고 주어진다. ($2 \leq V \leq 400$, $0 \leq E \leq V(V-1)$) 다음 E개의 줄에는 각각 세 개의 정수 a, b, c가 주어진다. a번 거점에서 b번 거점으로 가는 거리가 c인 도로가 있다는 의미이다. ($a \rightarrow b$ 임에 주의) 거리는 10,000 이하의 자연수이다. (a, b) 쌍이 같은 도로가 여러 번 주어지지 않는다.

```
3 4
1 2 1
3 2 1
1 3 5
2 3 2
```

출력

첫째 줄에 최소 사이클의 도로 길이의 합을 출력한다. 훈련 경로를 찾는 것이 불가능한 경우에는 -1을 출력한다.

```
3
```


1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

2) 플로이드-워셜 알고리즘 (Floyd-Warshall Algorithm)

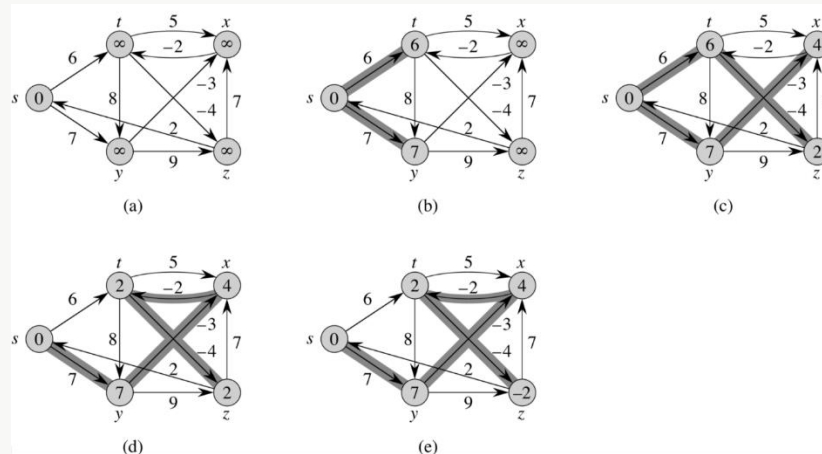
```
import sys
v, e = map(int, input().split())
inf = 100000000
s = [[inf] * v for i in range(v)]
for i in range(e):
    a, b, c = map(int, input().split())
    s[a - 1][b - 1] = c
for k in range(v):
    for i in range(v):
        for j in range(v):
            if s[i][j] > s[i][k] + s[k][j]:
                s[i][j] = s[i][k] + s[k][j]
result = inf
for i in range(v):
    result = min(result, s[i][i])
if result == inf:
    print(-1)
else:
    print(result)
```

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

3) 벨만-포드 알고리즘 (Bellman-Ford-Moore Algorithm)

음수 싸이클 없는 그래프에서 한정점 $\rightarrow$ 모든 정점 최단경로 및 거리

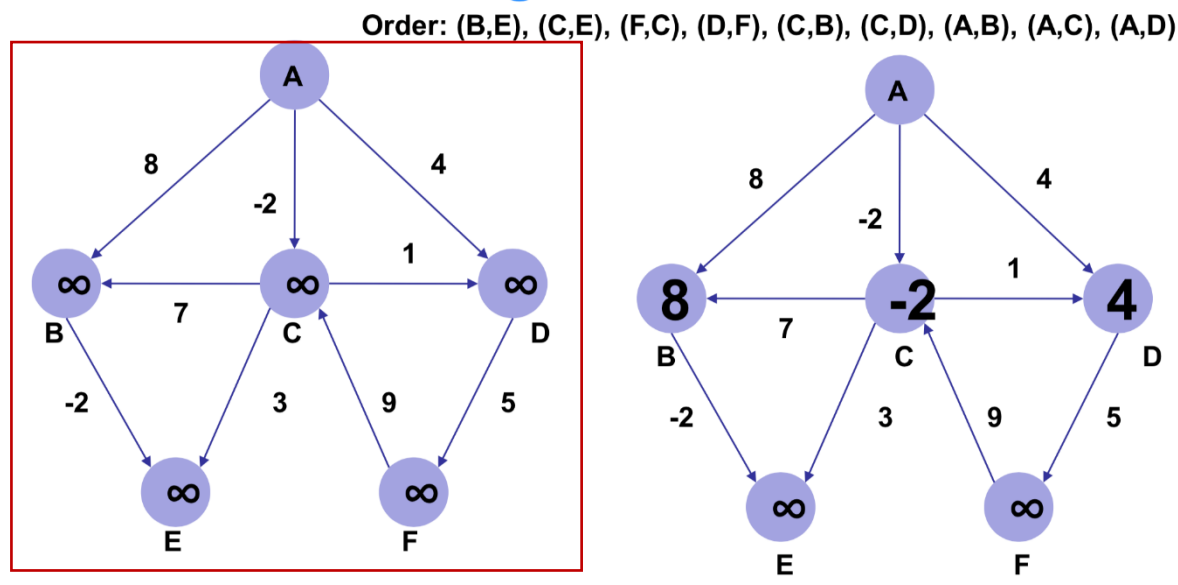
시작노드 s 에서 v 에 이르는 최단경로는 s 에서 u 까지의 최단경로에 u 에서 v 사이의 가중치(거리)를 더한 값



1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

3) 벨만-포드 알고리즘 (Bellman-Ford-Moore Algorithm)

벨만-포드 알고리즘의 동작 과정



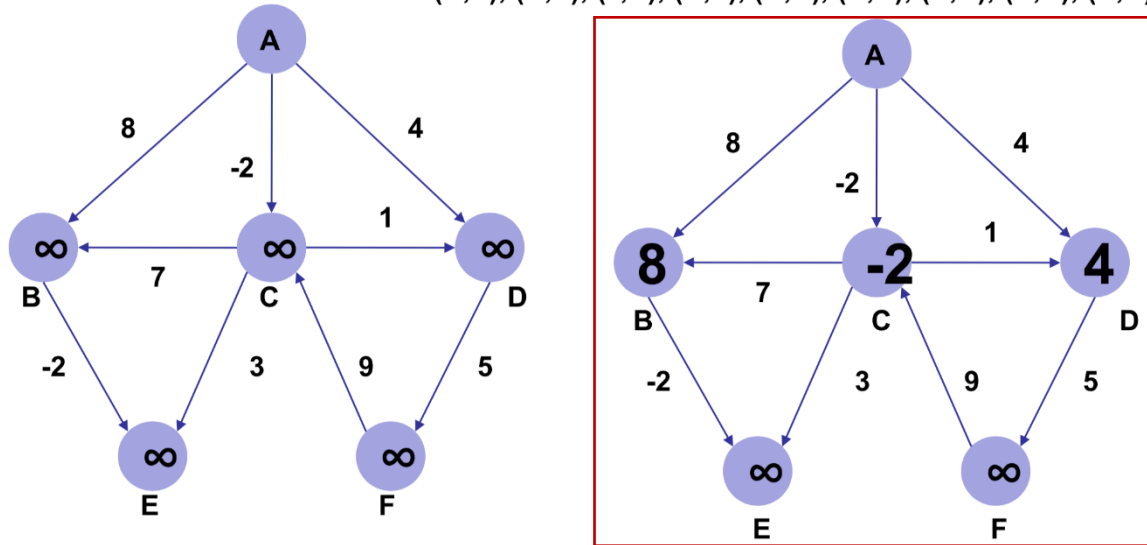
우선 시작노드 A를 제외한 모든 노드의 거리를 무한대로 설정
(edge relaxation 순서는 order를 참조)

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

3) 벨만-포드 알고리즘 (Bellman-Ford-Moore Algorithm)

벨만-포드 알고리즘의 동작 과정

Order: (B,E), (C,E), (F,C), (D,F), (C,B), (C,D), (A,B), (A,C), (A,D)



우선 (B,E)를 보면. 무한대-2=무한대이므로 업데이트할 필요가 없고 (C,E), (F,C), (D,F), (C,B) 또한 마찬가지이다. (A,B)의 경우 시작노드 A에서 B에 이르는 거리가 8이고, 이는 기존 거리(무한대)보다 작으므로 8을 B에 기록한다.

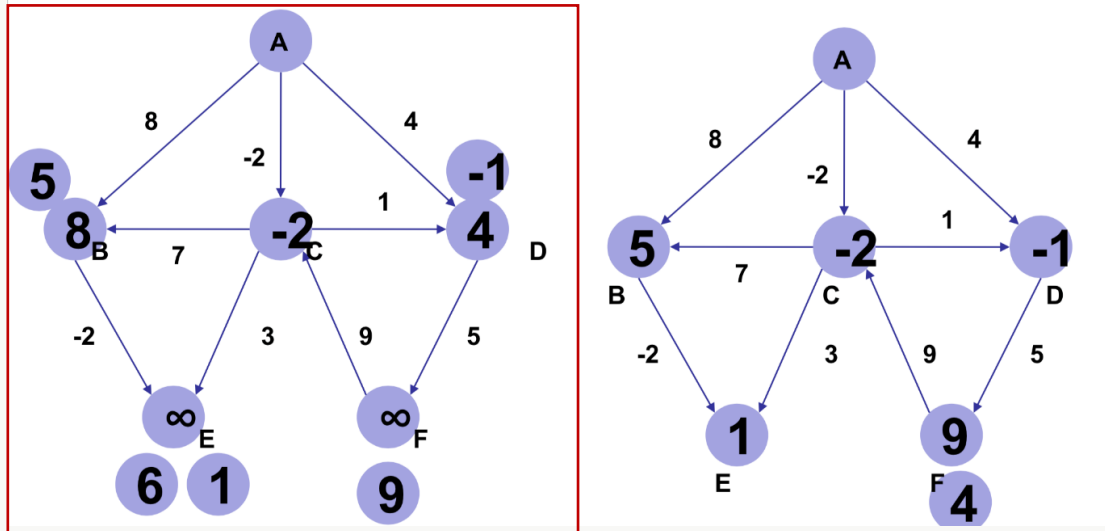
마찬가지로 C,D도 각각 -2, 4로 기록하고 그래프 모든 엣지에 대한 첫번째 edge relaxation이 종료

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

3) 벨만-포드 알고리즘 (Bellman-Ford-Moore Algorithm)

벨만-포드 알고리즘의 동작 과정

Order: (B,E), (C,E), (F,C), (D,F), (C,B), (C,D), (A,B), (A,C), (A,D)



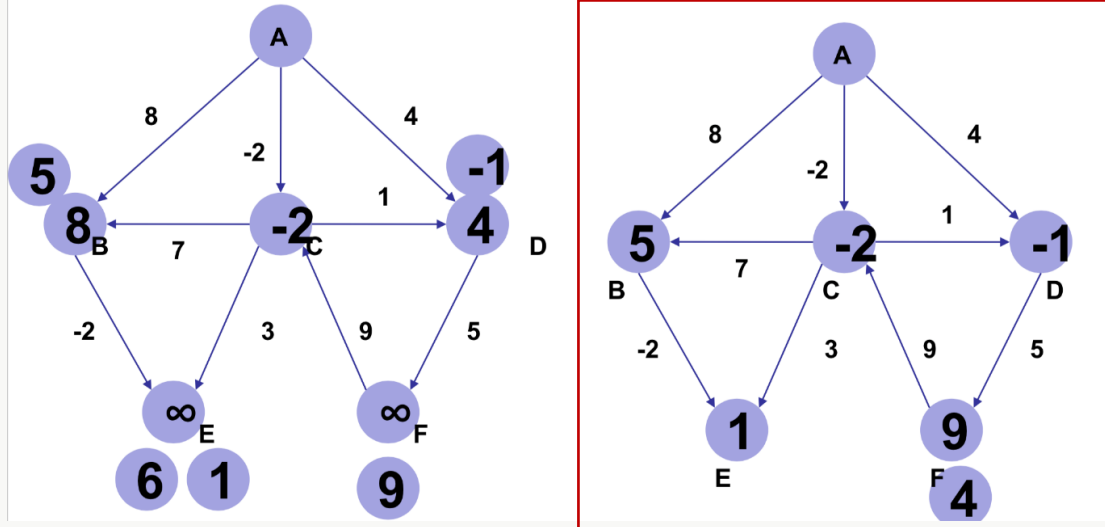
(B,E)의 경우 $8-2=6$ 이고 이는 기존 거리(무한대)보다 작으므로 6을 E에 기록
(C,E)의 경우 $-2+3=1$ 이고 이는 기존 거리(6)보다 작으므로 1을 E에 기록
(F,C)의 경우 무한대+9=무한대이므로 업데이트할 필요가 없다.
(D,F)의 경우 $4+5=9$ 이고 이는 기존 거리(무한대)보다 작으므로 9를 F에 기록
(C,B)의 경우 $-2+7=5$ 이고 이는 기존 거리(8)보다 작으므로 5를 B에 기록
(C,D)의 경우 $-2+1=-1$ 이고 이는 기존 거리(4)보다 작으므로 -1을 DD에 기록
(A,B)의 경우 $0+8=8$ 이고 이는 기존 거리(5)보다 크므로 업데이트할 필요가 없다.
(A,C)의 경우 $0-2=-2$ 이고 이는 기존 거리(-2)와 같으므로 업데이트할 필요가 없다.
(A,D)의 경우 $0+4=4$ 이고 이는 기존 거리(-1)보다 크므로 업데이트할 필요가 없다.
이로써 그래프 모든 엣지에 대한 두번째 edge relaxation이 종료

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

3) 벨만-포드 알고리즘 (Bellman-Ford-Moore Algorithm)

벨만-포드 알고리즘의 동작 과정

Order: (B,E), (C,E), (F,C), (D,F), (C,B), (C,D), (A,B), (A,C), (A,D)



(B,E)의 경우 $5-2=3$ 이고 이는 기존 거리(1)보다 크므로 업데이트할 필요가 없다. 이는 (C,E), (F,C) 또한 마찬가지이다.

(D,F)의 경우 $-1+5=4$ 이고 이는 기존 거리(9)보다 작으므로 4를 FF에 기록

(C,B), (C,D), (A,B), (A,C), (A,D)는 모두 기존 거리보다 크므로 업데이트할 필요가 없다. 이로써 그래프 모든 엣지에 대한 세번째 edge relaxation이 끝났습니다.

벨만-포드 알고리즘은 시작노드를 제외한 전체 노드수 만큼의 edge relaxation을 수행해야 한다. (위 예시의 경우 총 5회 반복 수행) 그런데 네번째 edge relaxation부터는 거리 정보가 바뀌지 않으므로 생략

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

3) 벨만-포드 알고리즘 (Bellman-Ford-Moore Algorithm)

벨만-포드 알고리즘의 특징

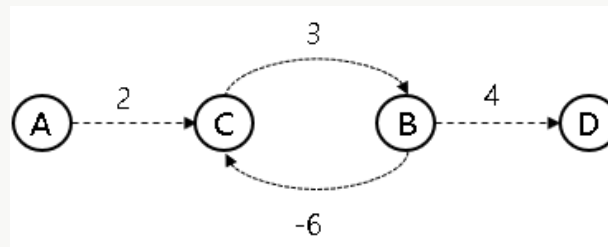
- 음의 가중치를 가지는 간선도 가능하다.
- 음의 사이클의 존재 여부를 확인할 수 있다.
- 최단거리를 구하기 위해서 $V-1$ 번 E 개의 모든 간선을 확인한다.
- 음의 사이클 존재 여부를 확인하기 위해서 한 번 더 E 개의 간선을 확인한다.
- 총 연산횟수는 $V \cdot E$ 이다. 따라서 시간복잡도는 $O(VE)$ 이다.
- V 번째 모든 간선을 확인하였을 때 거리 배열이 갱신되었다면, 그래프 G 는 음의 사이클을 가지는 그래프이다.
- 그래프 모든 엣지에 대해 edge relaxation을 시작노드를 제외한 전체 노드 수만큼 반복 수행한 뒤, 마지막으로 그래프 모든 엣지에 대해 edge relaxation을 1번 수행해 준다. 이때 한번이라도 업데이트가 일어난다면 음의 사이클이 존재한다는 뜻이 되어서 결과를 구할 수 없다는 의미의 **false**를 반환하고 함수를 종료한다

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

3) 벨만-포드 알고리즘 (Bellman-Ford-Moore Algorithm)

벨만-포드 알고리즘의 특징

음의 가중치 자체가 문제가 되는 것은 아니지만 아래와 같이 음의 값이 누적되는 사이클을 형성하면 문제가 된다.



A → D로 최단 거리 비용을 생각해보자.

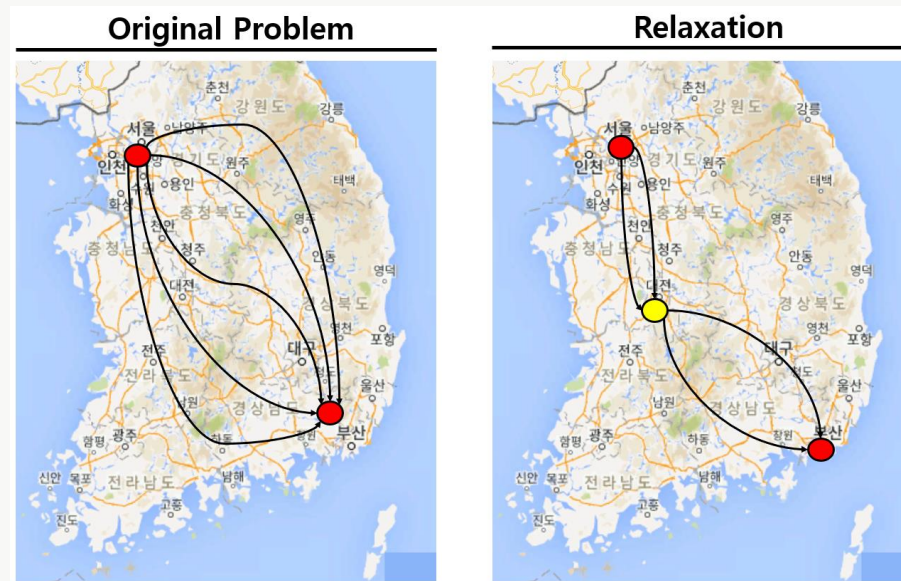
cost: 2 → 3 → -6 → 3 → -6 → 3 → -6 ... 무한히 최소 거리가 작아진다. 그렇기에 벨만 포드 알고리즘에서는 음의 값이 누적되는 사이클이 존재하는 경우, 의미 없는 값을 반환한다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

3) 벨만-포드 알고리즘 (Bellman-Ford-Moore Algorithm)

edge relaxation

relaxation이란 문제를 해결함에 있어서 조금 더 쉬운 문제로 풀고 이를 토대로 원래의 해를 찾는 방식



예) 서울에서 부산까지 가는 경로에는 굉장히 많은 도시들이 있다. 최적의 경로를 생각할 때, 중간에 대전을 끼고 서울-대전, 대전-부산으로 가는 경로를 생각한다면, 문제는 조금 더 쉽게 풀릴 수 있다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

3) 벨만-포드 알고리즘 (Bellman-Ford-Moore Algorithm)

```
def bf(start):
    dis[start] = 0 # 시작 지점 초기화

    # 매 반복마다 모든 간선 확인
    # 음의 간선 사이클 존재 유무가 필요하면 n번과 return 처리
    # 필요 없다면 n-1번과 리턴 처리는 필요 없음 dis 테이블만 필요함
    for i in range(n + 1):
        # 모든 간선 확인
        for j in range(m):
            current = edges[j][0]
            next_node = edges[j][1]
            cost = edges[j][2]
            # 시작위치에서 현재 노드까지 이동이 가능하면서
            # 현재 간선을 거쳐서 다른 노드로 이동하는 거리가 더 짧은경우
            if dis[current] != INF and dis[next_node] > cost + dis[current]:
                dis[next_node] = dis[current] + cost
            # 싸이클 유무 확인을 위해 n번 돌렸을 때
            # 최단 거리 갱신이 발생하면 음의 사이클이 존재
            if i == n - 1:
                return True
    return False

# 노드, 간선 개수
n, m = map(int, input().split())

edges = []
dis = [INF] * (n + 1) # 최단 거리 테이블

# 간선 정보
for _ in range(m):
    a, b, c = map(int, input().split())
    edges.append((a, b, c))

cycle = bf(1)
if cycle: # 음의 사이클 발생
    print(-1)
else:
    # 1번 노드에서 시작했으니 다른 노드로 가기 위한 최단 거리 출력
    for i in range(2, n + 1):
        if dis[i] == INF:
            print(-1)
        else:
            print(dis[i])
```

```
6 8
4 5 3
2 4 0
1 4 4
1 2 1
5 6 1
3 2 6
3 3 4
3 2 6
1
-1
1
4
5
```

```
4 4
1 2 1
2 3 2
3 2 -4
3 4 3
-1
```

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

3) 벨만-포드 알고리즘 (Bellman-Ford-Moore Algorithm)

때는 2020년, 국방이는 미래나라의 한 국민이다. 미래나라에는 N 개의 지점이 있고 N 개의 지점 사이에는 M 개의 도로와 W 개의 웜홀이 있다. (단 도로는 방향이 없으며 웜홀은 방향이 있다.) 웜홀은 시작 위치에서 도착 위치로 가는 하나의 경로인데, 특이하게도 도착을 하게 되면 시작을 하였을 때보다 시간이 뒤로 가게 된다. 웜홀 내에서는 시계가 거꾸로 간다고 생각하여도 좋다.

시간 여행을 매우 좋아하는 백준이는 한 가지 궁금증에 빠졌다. 한 지점에서 출발을 하여서 시간여행을 하기 시작하여 다시 출발을 하였던 위치로 돌아왔을 때, 출발을 하였을 때보다 시간이 되돌아가 있는 경우가 있는지 없는지 궁금해졌다. 여러분은 백준이를 도와 이런 일이 가능한지 불가능한지 구하는 프로그램을 작성하여라.

입력

첫 번째 줄에는 테스트케이스의 개수 $TC(1 \leq TC \leq 5)$ 가 주어진다. 그리고 두 번째 줄부터 TC 개의 테스트케이스가 차례로 주어지는데 각 테스트케이스의 첫 번째 줄에는 지점의 수 $N(1 \leq N \leq 500)$, 도로의 개수 $M(1 \leq M \leq 2500)$, 웜홀의 개수 $W(1 \leq W \leq 200)$ 이 주어진다. 그리고 두 번째 줄부터 $M+1$ 번째 줄에 도로의 정보가 주어지는데 각 도로의 정보는 S, E, T 세 정수로 주어진다. S 와 E 는 연결된 지점의 번호, T 는 이 도로를 통해 이동하는데 걸리는 시간을 의미한다. 그리고 $M+2$ 번째 줄부터 $M+W+1$ 번째 줄까지 웜홀의 정보가 S, E, T 세 정수로 주어지는데 S 는 시작 지점, E 는 도착 지점, T 는 줄어드는 시간을 의미한다. T 는 10,000보다 작거나 같은 자연수 또는 0이다.

두 지점을 연결하는 도로가 한 개보다 많을 수도 있다. 지점의 번호는 1부터 N 까지 자연수로 중복 없이 매겨져 있다.

```
2
3 3 1
1 2 2
1 3 4
2 3 1
3 1 3
NO
3 2 1
1 2 3
2 3 4
3 1 8
YES
```

출력

TC 개의 줄에 걸쳐서 만약에 시간이 줄어들면서 출발 위치로 돌아오는 것이 가능하면 YES, 불가능하면 NO를 출력한다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

3) 벨만-포드 알고리즘 (Bellman-Ford-Moore Algorithm)

```
import sys
#input = sys.stdin.readline
INF = int(1e9)
def bellmanFord():
    global isPossible

    for i in range(1, N + 1):
        for j in range(1, N + 1):
            for wei, vec in adjList[j]:
                if dist[vec] > wei + dist[j]:
                    dist[vec] = wei + dist[j]
            if i == N:
                isPossible = False

T = int(input())

for _ in range(T):
    N, M, W = map(int, input().split())
    dist = [INF for _ in range(N + 1)]

    adjList = [[] for _ in range(N + 1)]

    for _ in range(M):
        S, E, T = map(int, input().split())

        adjList[S].append((T, E))
        adjList[E].append((T, S))

    for _ in range(W):
        S, E, T = map(int, input().split())
        adjList[S].append((-T, E))

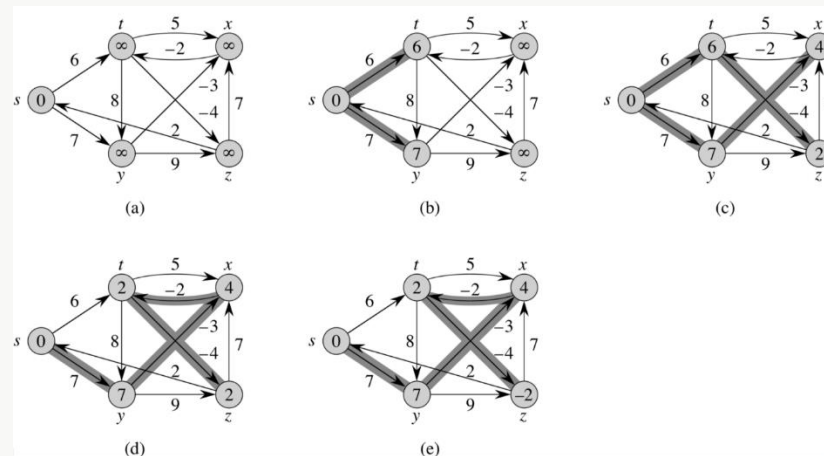
    isPossible = True
    bellmanFord()
    print("NO" if isPossible else "YES")
```

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

A* 알고리즘은 시작 노드만을 지정해 다른 모든 노드에 대한 최단 경로를 파악하는 다익스트라 알고리즘과 다르게 시작 노드와 목적지 노드를 분명하게 지정해 이 두 노드 간의 최단 경로를 파악

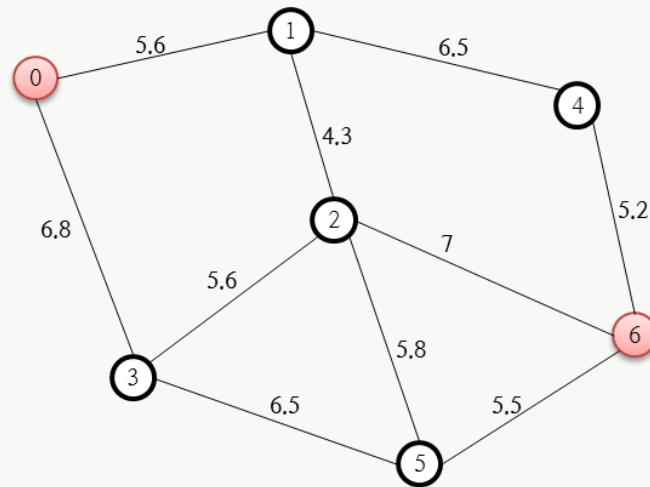
A* 알고리즘은 휴리스틱 추정값을 통해 알고리즘을 개선할 수 있는데 이러한 휴리스틱 추정값을 어떤 방식으로 제공하느냐에 따라 얼마나 빨리 최단 경로를 파악할 수 있느냐가 결정



1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

A* 알고리즘의 동작 과정

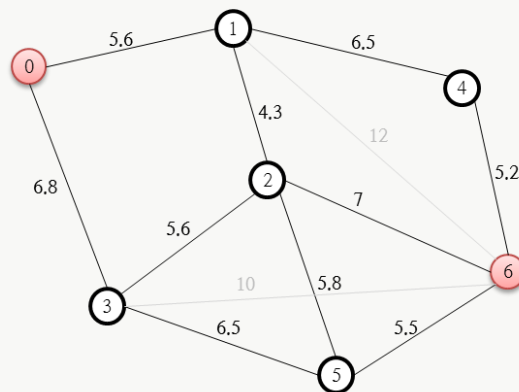


시작점인 0번 노드에서 목적지인 6번 노드로 가는 최단 경로를 A* 알고리즘으로 분석하고자 하는 것인데, 각 노드 사이에 연결된 링크에 붙은 숫자는 노드 사이를 이동하는데 소요되는 비용(경비, Cost)이다. 위의 경우 거리 값이다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

A* 알고리즘의 동작 과정



O =

Node ID	1	3
F Score	17.6	16.8
G Score	5.6	6.8
H Score	12	10
Parent Node	0	0

C =

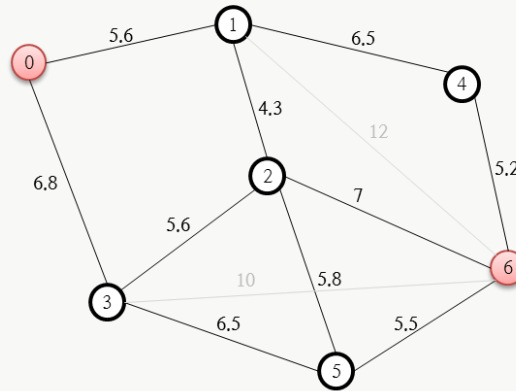
Node ID	0
F Score	0
G Score	0
H Score	0
Parent Node	-

위의 그림에서 보면 저장소로 O와 C가 있는데, O는 열린 목록(Open List), C는 닫힌 목록(Close List)이다. 열린 목록인 O 저장소에는 최단 경로를 분석하기 위한 상태 값들이 계속 갱신되며, C 저장소는 처리가 완료된 노드를 담아 두기 위한 목적으로 사용된다. 이러한 O와 C의 저장소를 기반으로 0번 노드에서 6번 노드까지의 최단 경로를 산출해 보도록 하겠다

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

A* 알고리즘의 동작 과정



O =

Node ID	1	3
F Score	17.6	16.8
G Score	5.6	6.8
H Score	12	10
Parent Node	0	0

C =

Node ID	0
F Score	0
G Score	0
H Score	0
Parent Node	-

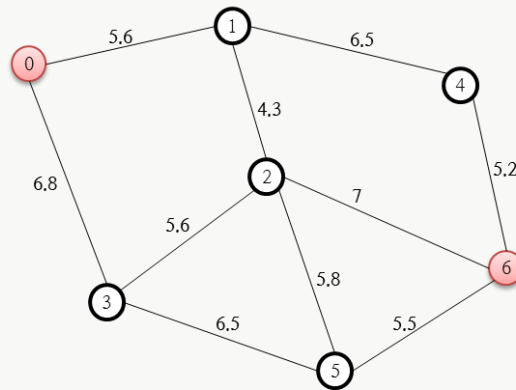
먼저 출발 노드인 0을 담은 목록인 C 목록에 집어넣는다. 그리고 이 0번 노드와 연결된 노드는 1번과 3번 노드를 열린 목록인 O 저장소에 추가한다. 추가할 때 F, G, H, Parent Node값도 함께 추가해야 하는데, 먼저 $F = G + H$ 이다. G는 시작 노드에서 해당 노드까지의 실제 소요 경비 값이고, H는 휴리스틱 추정 값으로 해당 노드에서 최종 목적지까지 도달하는데 소요될 것이라고 추정되는 값이다. Parent Node는 해당 노드에 도달하기 직전에 거치는 노드 번호이다. 먼저 1번 노드에 대한 F, G, H, Parent Node를 살펴보겠다. 출발점인 0번 노드로부터 시작했으므로, 1번 노드의 Parent Node는 0번 노드이다. 그리고 G 값은 0번 노드에서 1번 노드까지의 거리 비용 값인 5.6이다. H 값을 추정하기 위한 기준이 필요한데, 이 추정 값에 대한 기준을 1번 노드에서 목적지인 6번 노드까지의 직선 거리로 하기로 정하고 측정을 하니(줄자로 재든, 좌표가 있다면 피타고라스 정리를 통해 두 좌표 사이의 거리를 계산하든 하여 얻을 수 있음) 12로 산출되므로 H는 12가 된다.

$F = G + H$ 이므로 $5.6 + 12$ 인 17.6이 된다. 3번에 대한 F, G, H, Parent Node 역시 이와 동일하게 결정할 수 있다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

A* 알고리즘의 동작 과정



O =

Node ID	1	2	5
F Score	17.6	19.4	18.8
G Score	5.6	12.4	13.3
H Score	12	7	5.5
Parent Node	0	3	3

C =

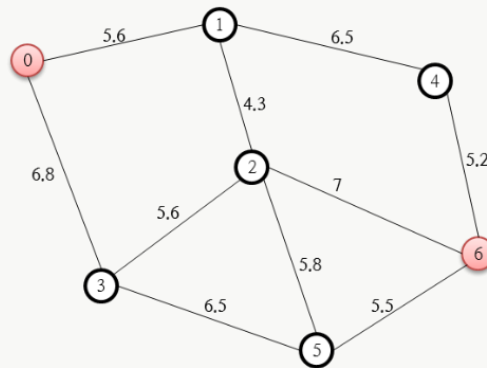
Node ID	0	3
F Score	0	16.8
G Score	0	6.8
H Score	0	10
Parent Node	-	0

O 리스트 중 F 값이 가장 작은 노드는 3번인데, 이 노드 3번을 C 리스트에 추가하고 3번 노드와 연결된 0, 2, 5 중 닫힌 목록에 존재하지 않는 2, 5번 노드에 대해 열린 목록에 추가한다. 2, 5에 대한 F, G, H, Parent Node를 계산해 기록한다. 먼저 2번 노드에 대해 계산해 보면, 2번 노드에 대한 G 값은 바로 직전 노드에 소요되는 비용(6.8)에 3번 노드에서 2번 노드까지 도달하기 위한 비용인 5.6을 합한 값인 12.4가 된다. 그리고 H 값은 2번 노드에서 목적지인 6번지에 대한 거리 값인 7이 된다. 5번 노드에 대한 것도 이와 동일하게 계산한다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

A* 알고리즘의 동작 과정



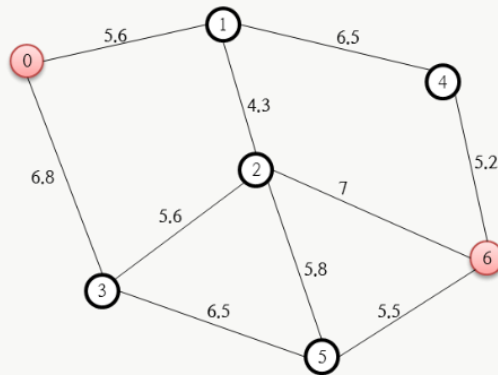
	2																						
	19.4																						
	12.4																						
	7																						
	3																						
	↓																						
○ =	<table><tr><td>Node ID</td><td>2</td><td>5</td><td>4</td></tr><tr><td>F Score</td><td>16.9</td><td>18.8</td><td>17.3</td></tr><tr><td>G Score</td><td>9.9</td><td>13.3</td><td>12.1</td></tr><tr><td>H Score</td><td>7</td><td>5.5</td><td>5.2</td></tr><tr><td>Parent Node</td><td>1</td><td>3</td><td>1</td></tr></table>	Node ID	2	5	4	F Score	16.9	18.8	17.3	G Score	9.9	13.3	12.1	H Score	7	5.5	5.2	Parent Node	1	3	1		
Node ID	2	5	4																				
F Score	16.9	18.8	17.3																				
G Score	9.9	13.3	12.1																				
H Score	7	5.5	5.2																				
Parent Node	1	3	1																				
C =	<table><tr><td>Node ID</td><td>0</td><td>3</td><td>1</td></tr><tr><td>F Score</td><td>0</td><td>16.8</td><td>17.6</td></tr><tr><td>G Score</td><td>0</td><td>6.8</td><td>5.6</td></tr><tr><td>H Score</td><td>0</td><td>10</td><td>12</td></tr><tr><td>Parent Node</td><td>-</td><td>0</td><td>0</td></tr></table>	Node ID	0	3	1	F Score	0	16.8	17.6	G Score	0	6.8	5.6	H Score	0	10	12	Parent Node	-	0	0		
Node ID	0	3	1																				
F Score	0	16.8	17.6																				
G Score	0	6.8	5.6																				
H Score	0	10	12																				
Parent Node	-	0	0																				

열린 목록(O 저장소) 중 F 값이 가장 작은(최소인) 1번 노드를 닫힌 목록에 추가한다. 그리고 이 1번 노드와 연결된 2, 4번 노드 중 닫힌 목록(C 저장소)에 존재하지 않는 것에 대해 다시 F, G, H, Parent Node를 계산한다. 이 상태에서 4번 노드는 열린 목록에 없었던 것이기에 그냥 F, G, H, Parent Node를 이미 앞서 설명했던 방식으로 계산해 추가하면 그만이지만 2번 노드는 전 단계에서 이미 추가되어 있었다. 이렇게 전 단계에서 추가된 G 값이 새롭게 계산된 G 값보다 크다면 새롭게 계산된 F, G, H, Parent Node 값으로 변경해줘야 하며 위의 그림이 이러한 변경을 나타내고 있다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

A* 알고리즘의 동작 과정



O =

Node ID	5	4	6
F Score	18.8	17.3	16.9
G Score	13.3	12.1	16.9
H Score	5.5	5.2	0
Parent Node	3	1	2

C =

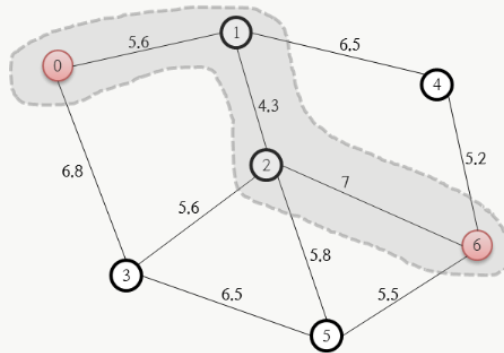
Node ID	0	3	1	2
F Score	0	16.8	17.6	16.9
G Score	0	6.8	5.6	9.9
H Score	0	10	12	7
Parent Node	-	0	0	1

열린 목록 중 F가 최소인 노드는 2번 노드이고, 이 2번 노드를 닫힌 목록에 추가한다. 그리고 2번 노드와 연결된 1, 3, 5, 6번 노드 중 닫힌 목록에 존재하지 않는 5, 6번 노드에 대한 F, G, H Parent Node 값을 계산한다. 5번 노드의 경우 새로운 G 값이 기존 값보다 크므로 변경하지 않고 6번 노드에 대한 값들만을 계산해 추가한다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

A* 알고리즘의 동작 과정



O =

Node ID	5	4
F Score	18.8	17.3
G Score	13.3	12.1
H Score	5.5	5.2
Parent Node	3	1

C =

Node ID	0	3	1	2	6
F Score	0	16.8	17.6	16.9	16.9
G Score	0	6.8	5.6	9.9	16.9
H Score	0	10	12	7	0
Parent Node	-	0	0	1	2

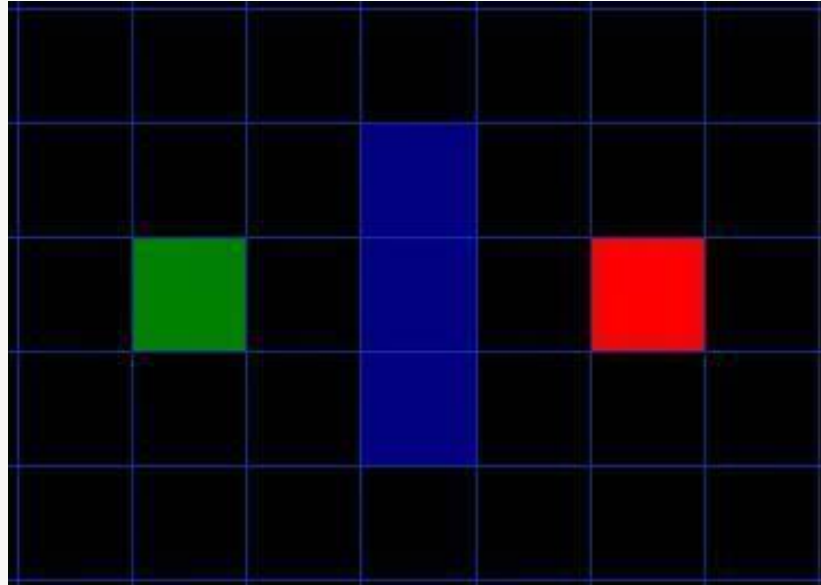
열린 목록 중 F가 최소인 노드는 6번 노드인데, 이 6번 노드를 닫힌 목록에 추가한다. 그런데 이 6번 노드는 최종 목적지 노드이므로 A* 알고리즘은 종료된다.

여기까지 만들어진 닫힌 목록(C 저장소)을 토대로 0번 노드에서 6번 노드까지의 최단 경로를 파악할 수 있다. 6번 노드의 Parent Node는 2번이고, 2번 노드의 Parent Node는 1번이며, 1번 노드의 Parent Node는 0번이므로 최단 경로는 6번 노드←2번 노드←1번 노드←0번 노드가 된다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

A* 알고리즘의 탐색 과정



간단하게 우리가 길을 찾아야 할 지역을 탐색해보도록 하겠다. A지점에서 B지점으로 가는 것으로 목표가 정해졌다. 두 포인트 사이에 벽이 있다. 이 가정은 아래 그림과 같다. 초록색은 시작포인트 A, 빨간색은 엔딩포인트 B이다. 그리고 파란색으로 두 지점 사이에 벽이 쳐져있다.

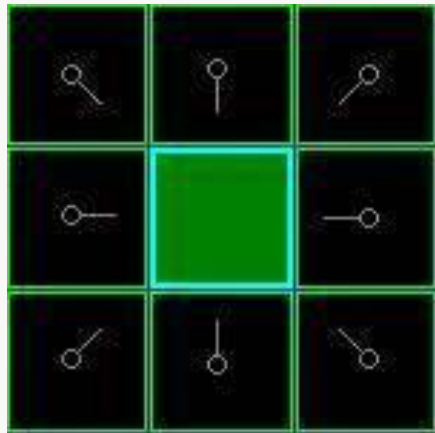
1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

A* 알고리즘의 탐색 과정

A점으로부터 B점까지 인접 사각형을 확인하면서 길을 만들어나가겠다. 아래의 순서로 스타트를 끊어보자.

1. 시작점 A를 '열린 목록'에 넣어주자. 이 목록은 일종의 장바구니와 같다. 지금은 시작점만 들어있지만 점점 많아질 것이다.
2. 시작점에 인접한 벽, 물, 또는 위험 지형은 무시하고 지나갈 수 있는 사각형을 '열린 목록'에 넣어주자. 이 사각형들은 시작점 A를 부모로 지정한다. ('부모 사각형'은 길을 다 찾고 거슬러 올라갈 때 중요한 것만 일단 기억하라.)
3. '열린 목록'에서 시작점 A 사각형을 삭제하고 다시 볼 필요 없는 '닫힌 목록'에 추가해주자.



녹색 사각형은 스타트 포인트이다. 하늘색 선으로 둘러싸여 있다는 것은 이제 그 사각형은 '닫힌 목록'에 버려졌으니 다시 볼 필요 없다는 뜻이다. 인접한 8개의 사각형은 밝은 녹색으로 윤곽선이 그려져 있다. '열린 목록'에 들어간다는 뜻이다. 그리고 각각 회색 포인트로 부모 노드를 가리키고 있다. 우리는 '열린 목록'에 있는 사각형들 중에 하나를 선택해서 위의 순서로 반복 처리를 할 것이다. 그럼 어떤 사각형을 선택해야할까? 바로 가장 작은 비용 F를 가진 사각형이다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

A* 알고리즘의 탐색 과정

$$F = G + H$$

G - 시작점 A로부터 현재 사각형까지의 경로를 따라 이동하는데 소요되는 비용이다.

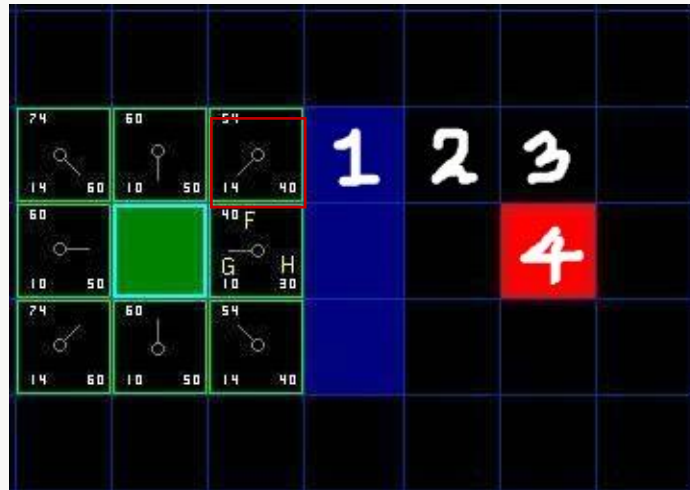
H - 현재 사각형에서 목적지 B까지의 예상 이동 비용이다. 사이에 벽, 물 등으로 인해 실제 거리는 알지 못한다. 그들을 무시하고 예상 거리를 산출합니다. 여러 방법이 있지만, 이 글에서는 대각선 이동을 생각하지 않고, 가로 또는 세로로 이동하는 비용만 계산한다.

F - 현재까지 이동하는데 걸린 비용과 예상 비용을 합친 총 비용이다.

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

A* 알고리즘의 탐색 과정



G는 스타팅 포인트로부터 대각선 방향에 있으므로 14이다. (G는 시작 위치로부터 현재 위치의 실제 거리를 알기 때문에 대각선을 포함하여 계산한다.) H는 4칸 이동이므로 40이다. F는 총 합이니 54가 된다.

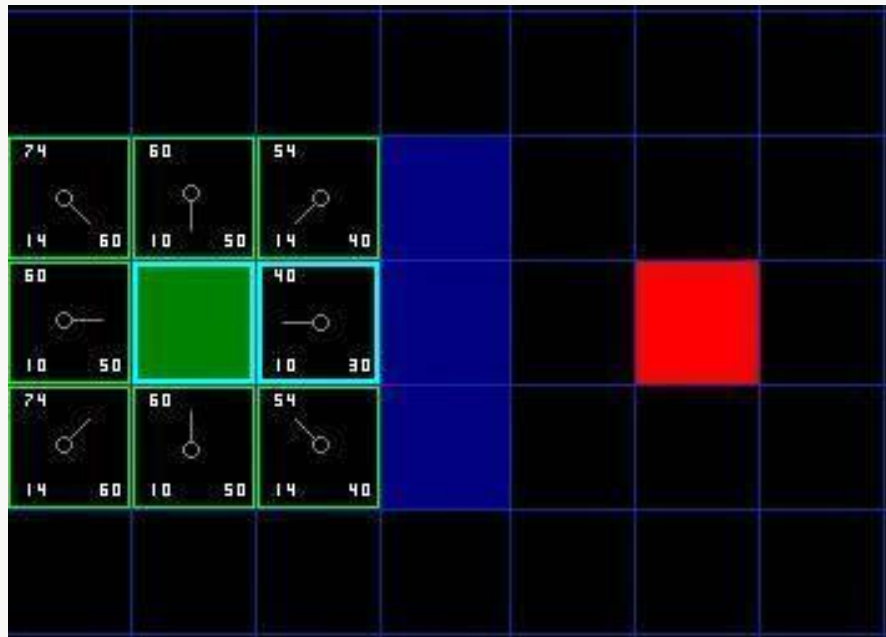
1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

A* 알고리즘의 탐색 과정

계속 탐색하기 위해서, '열린 목록'에서 **가장 작은 F 비용을 가지고 있는 사각형을 선택한다**. 그리곤 아래의 스텝을 따라주면 된다.

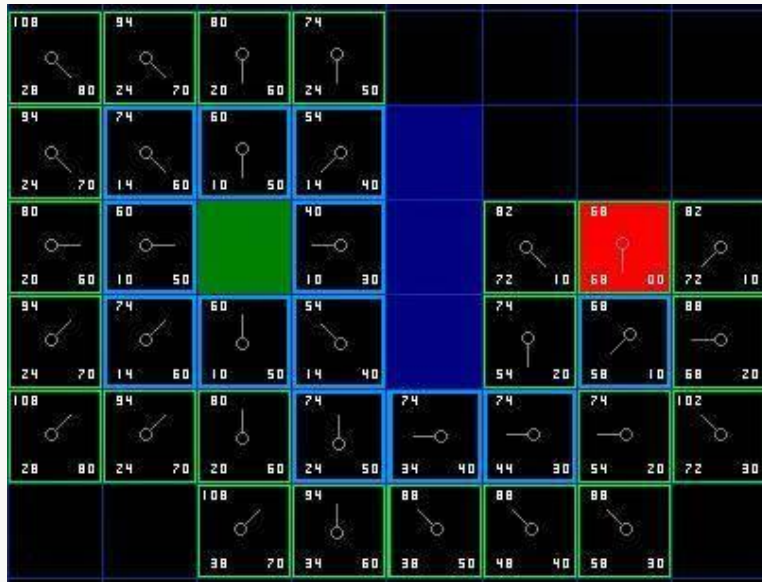
1. 선택한 사각형을 '열린 목록'에서 빼고 '닫힌 목록'에 넣어주자.
2. 인접한 사각형을 확인한다. '닫힌 목록'에 있거나 벽은 무시하고 '열린 목록'에 사각형이 없다면 '열린 목록'에 추가해준다. 현재 사각형을 새로운 사각형의 '부모'로 지정한다.
3. 인접한 사각형이 이미 '열린 목록'에 있다면 해당 사각형의 비용이 더 좋은지 확인한다. 즉, 선택한 사각형과 비교하여 G점수가 어떤 것이 더 낮은지 확인한다. 그렇지 않으면 아무것도 하지 않는다. 하지만 해당 사각형의 G 비용이 더 낮은 경우 인접 사각형들의 부모를 새로운 사각형으로 바꾼다. 마지막으로, 그 사각형의 F와 G를 다시 계산



1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

A* 알고리즘의 탐색 과정



1. 열린 목록에서 가장 낮은 F 비용을 찾아 현재사각형으로 선택한다.
2. 이것을 열린 목록에서 꺼내 닫힌 목록으로 넣는다.
3. 선택한 사각형에 인접한 8 개의 사각형에 대해 탐색한다.

- 만약 인접한 사각형이 갈 수 없는 벽 이거나 그것이 '닫힌 목록' 상에 있다면 무시, 그렇지 않은 것은 다음을 계속한다.
 - 만약 이것이 '열린 목록'에 있지 않다면, 이것을 열린 목록에 추가하고, 이 사각형의 부모를 현재 사각형으로 만든다. 사각형의 F, G, H 비용을 기록한다.
 - 만약 이것이 이미 '열린 목록'에 있다면, G비용을 이용하여 이 사각형이 더 나은지 알아보고, 그것의 G비용이 더 작으면 그것이 더 나은 길이라는 것을 의미하므로 부모 사각형을 그 (G비용이 더 작은)사각형으로 바꾼다. 그리고 그 사각형의 G, F비용을 다시 계산한다.
- ㄹ. 그만해야 할 때
- 길을 찾는 중 목표사각형을 '열린 목록'에 추가했을 때
 - '열린 목록'이 비어있게 될 때(이 때는 목표사각형을 찾는데 실패한 것인데 이 경우 길이 없는 경우다.)

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

```
class Node:
    def __init__(self, parent=None, position=None):
        self.parent = parent
        self.position = position

        self.g = 0
        self.h = 0
        self.f = 0

    def __eq__(self, other):
        return self.position == other.position

def heuristic(node, goal, D=1, D2=2 ** 0.5): # Diagonal Distance
    dx = abs(node.position[0] - goal.position[0])
    dy = abs(node.position[1] - goal.position[1])
    return D * (dx + dy) + (D2 - 2 * D) * min(dx, dy)

def aStar(maze, start, end):
    # startNode와 endNode 초기화
    startNode = Node(None, start)
    endNode = Node(None, end)

    # openList, closedList 초기화
    openList = []
    closedList = []

    # openList에 시작 노드 추가
    openList.append(startNode)

    # endNode를 찾을 때까지 실행
    while openList:
        # 현재 노드 지정
        currentNode = openList[0]
        currentIdx = 0

        # 이미 같은 노드가 openList에 있고, f 값이 더 크면
        # currentNode를 openList안에 있는 값으로 교체
        for index, item in enumerate(openList):
            if item.f < currentNode.f:
                currentNode = item
                currentIdx = index
```

#####

```
# openList에서 제거하고 closedList에 추가
openList.pop(currentIdx)
closedList.append(currentNode)

# 현재 노드가 목적지면 current.position 추가하고
# current의 부모로 이동
if currentNode == endNode:
    path = []
    current = currentNode
    while current is not None:
        # maze 길을 표시하려면 주석 해제
        # x, y = current.position
        # maze[x][y] = 7
        path.append(current.position)
        current = current.parent
    return path[::-1] # reverse
```

```
children = []
# 인접한 xy좌표 전부
for newPosition in [(0, -1), (0, 1), (-1, 0), (1, 0), (-1, -1), (-1, 1), (1, -1), (1, 1)]:
```

```
# 노드 위치 업데이트
nodePosition = (
    currentNode.position[0] + newPosition[0], # X
    currentNode.position[1] + newPosition[1]) # Y
```

```
# 미로 maze index 범위 안에 있어야함
within_range_criteria = [
    nodePosition[0] > (len(maze) - 1),
    nodePosition[0] < 0,
    nodePosition[1] > (len(maze[0]) - 1),
    nodePosition[1] < 0,
]
```

```
if any(within_range_criteria): # 하나라도 true면 범위 밖임
    continue
```

```
# 장애물이 있으면 다른 위치 불러오기
if maze[nodePosition[0]][nodePosition[1]] != 0:
    continue
```

```
new_node = Node(currentNode, nodePosition)
children.append(new_node)
```

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

```
# 자식들 모두 loop
for child in children:

    # 자식이 closedList에 있으면 continue
    if child in closedList:
        continue

    # f, g, h 값 업데이트
    child.g = currentNode.g + 1
    child.h = ((child.position[0] - endNode.position[0]) **
                2) + ((child.position[1] - endNode.position[1]) ** 2)
    # child.h = heuristic(child, endNode) 다른 휴리스틱
    # print("position:", child.position) 거리 추정 값 보기
    # print("from child to goal:", child.h)

    child.f = child.g + child.h

    # 자식이 openList에 있고, g값이 더 크면 continue
    if len([openNode for openNode in openList
            if child == openNode and child.g > openNode.g]) > 0:
        continue

    openList.append(child)

def main():
    # 1은 장애물
    maze = [[0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]]

    start = (0, 0)
    end = (7, 6)

    path = aStar(maze, start, end)
    print(path)
if __name__ == '__main__':
    main()
```

[(0, 0), (1, 1), (2, 2), (3, 3), (4, 3), (5, 4), (6, 5), (7, 6)]

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

4) A* 알고리즘

3 x 3 보드에 8개의 조각이 있고 열린 슬롯에 조각을 밀어 넣을 수 있는 퍼즐이 있다. 게임의 목표는 조각을 무작위로 재배열한 후 보드의 숫자를 순서대로 맞추는 것이다.

8	5	3
	1	7
6	2	4



1	2	3
4	5	6
7	8	

얼마 동안 퍼즐을 가지고 놀다가 철수는 최소한의 단계로 모든 사례를 해결할 수 있다고 주장한다. 하지만 당신은 그를 믿지 않기 때문에 퍼즐을 최적으로 푸는 프로그램을 작성한다.

입력

입력의 첫 번째 줄은 테스트 케이스의 수 $1 \leq n \leq 100$ 이고 각 테스트 케이스의 입력은 각 라인의 3개 숫자 혹은 기호로 구성된다. 숫자는 1~8까지, 열린공간을 나타내는 #은 한 번 입력 가능하다.

2

123

4#5

786

123

456

87#

출력

각 테스트 케이스에 대해 퍼즐을 풀기 위한 최소 이동 수를 출력하거나, 수행할 수 없는 경우 불가능을 출력한다.

2

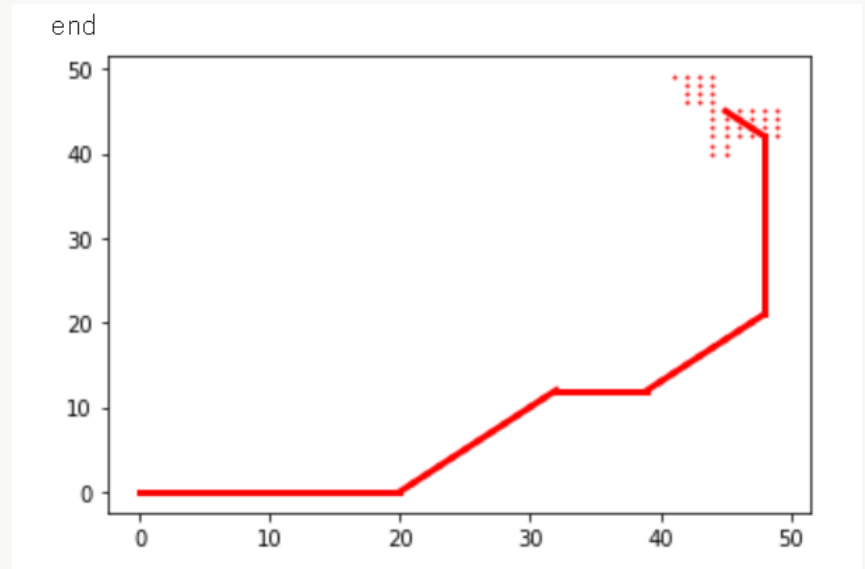
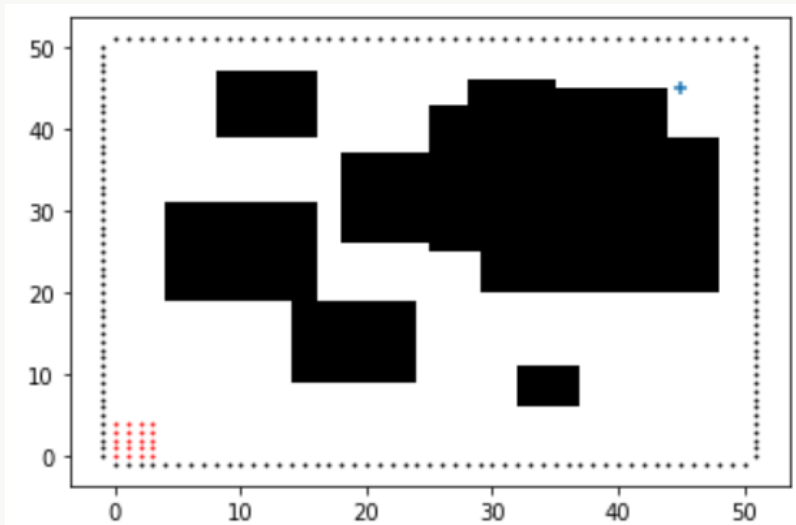
impossible

1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

5) 다익스트라 vs A*

A* 알고리즘은 Dijkstra 알고리즘과 유사한 점이 많다. 먼저 Dijkstra 알고리즘을 간단히 살펴보자

다익스트라 알고리즘의 pseudocode이다. Breadth-first Search 알고리즘과 비슷한 구조를 가지고 있음을 알 수 있다. 인접노드를 탐색하고 각 노드까지의 비용을 계산해서 최저 비용으로 갱신하는 방식으로 구현할 수 있다. 간단한 구현을 위해 인접 셀과 연결되어 있는 grid구조를 이용하겠다.



1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

5) 다익스트라 vs A*

A star알고리즘은 cost를 계산하는 부분에서 차이가 있다.

```
def heuristics(node):
```

```
    return sqrt((node[0] - goal[0])**2+(node[1] - goal[1])**2)
```

```
# dijkstra에 표시된 곳 내용 수정
```

```
deltaCost = dCost[i] + heuristics(next)
```

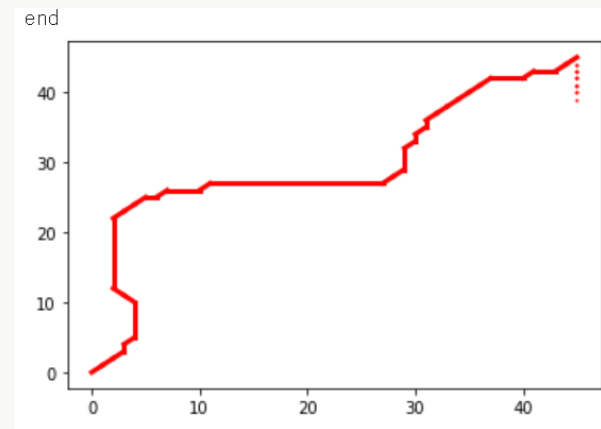
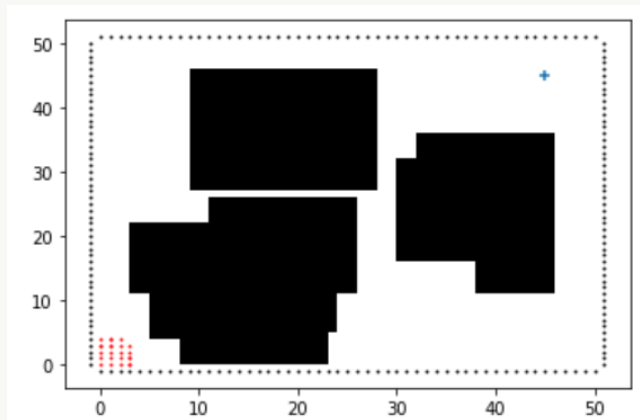
```
if cost[next[0]][next[1]] > cost[current[0]][current[1]] + deltaCost:
```

```
    Q.append(next)
```

```
    path[next[0]][next[1]] = current
```

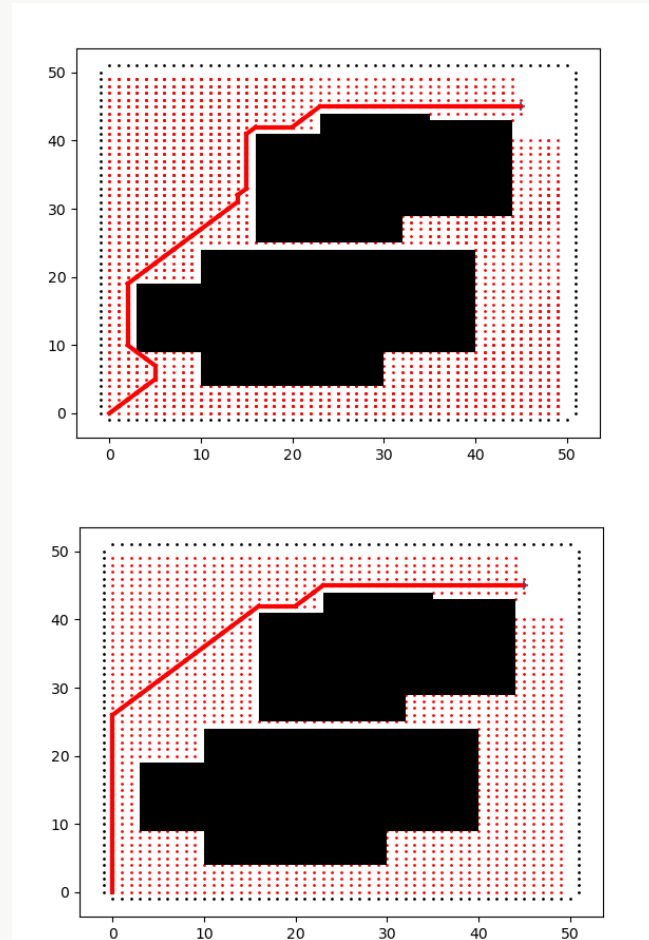
```
    cost[next[0]][next[1]] = cost[current[0]][current[1]] + deltaCost
```

heuristics 함수의 값이 cost에 추가 되어 계산되고 heuristics는 간단히 어림 짐작한 cost라고 볼 수 있다.



1. 차세대 전차를 위해 필요한 AI 알고리즘의 이해

5) 다익스트라 vs A*



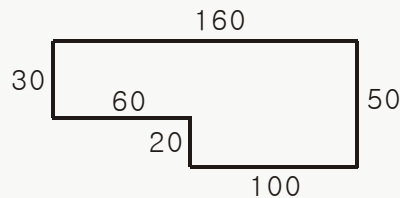
heuristic함수의 값이 오히려 좋지 않은 경로를 탐색할 수도 있다. A*의 성능은 heuristic함수가 얼마나 잘 설계되느냐로 성능이 결정된다 라고 볼 수 있다.

이 두 알고리즘의 단점은 Map이 클 경우 (Graph가 복잡한 경우)에 메모리소모가 크다는 점이 있겠고, 특히 A*의 경우 heuristic 함수의 의존도가 크다 라는 점을 들 수 있다.

아이스 브레이킹 #3

병력 분석

적의 거점 탈환을 위한 적 진지에 주둔하는 적 병력의 파악이 필요하다. 1의 넓이에 주둔하는 병력을 헤아린 다음, 진지의 넓이를 구하면 비례식을 이용하여 병력의 수를 대략적으로 구할 수 있다. 1m^2 의 넓이에 병력수는 파악했고, 이제 진지의 넓이만 구하면 된다. 형태는 Γ -자 모양이거나 Γ -자를 90도, 180도, 270도 회전한 모양(Γ , $\sqcap$, $\sqcup$ 모양)의 육각형이다.



예를 들어 위 그림과 같은 모양이라고 하자. 그림에서 오른쪽은 동쪽, 왼쪽은 서쪽, 아래쪽은 남쪽, 위쪽은 북쪽이다. 이 그림의 왼쪽 위 꼭지점에서 출발하여, 반시계방향으로 남쪽으로 30, 동쪽으로 60, 남쪽으로 20, 동쪽으로 100, 북쪽으로 50, 서쪽으로 160 이동하면 다시 출발점으로 되돌아가게 된다.

위 그림의 진지 면적은 6800 이다. 만약 1m^2 의 넓이에 병력수가 7 이라면, 적의 예상 병력수는 47600으로 계산된다.

1m^2 의 넓이에 병력수와, 진지를 이루는 육각형의 임의의 한 꼭지점에서 출발하여 반시계방향으로 둘레를 돌면서 지나는 변의 방향과 길이가 순서대로 주어진다. 적 진지의 병력수를 구하는 프로그램을 작성하시오.

첫 번째 줄에 1m^2 의 넓이에 병력을 나타내는 양의 정수참외발을 나타내는 육각형의 임의의 한 꼭지점에서 출발하여 반시계방향으로 둘레를 돌면서 지나는 변의 방향과 길이 (1 이상 500 이하의 정수) 가 둘째 줄부터 일곱 번째 줄까지 한 줄에 하나씩 순서대로 주어진다. 변의 방향에서 동쪽은 1, 서쪽은 2, 남쪽은 3, 북쪽은 4로 나타낸다.

입력

7
4 50
2 160
3 30
1 60
3 20
1 100

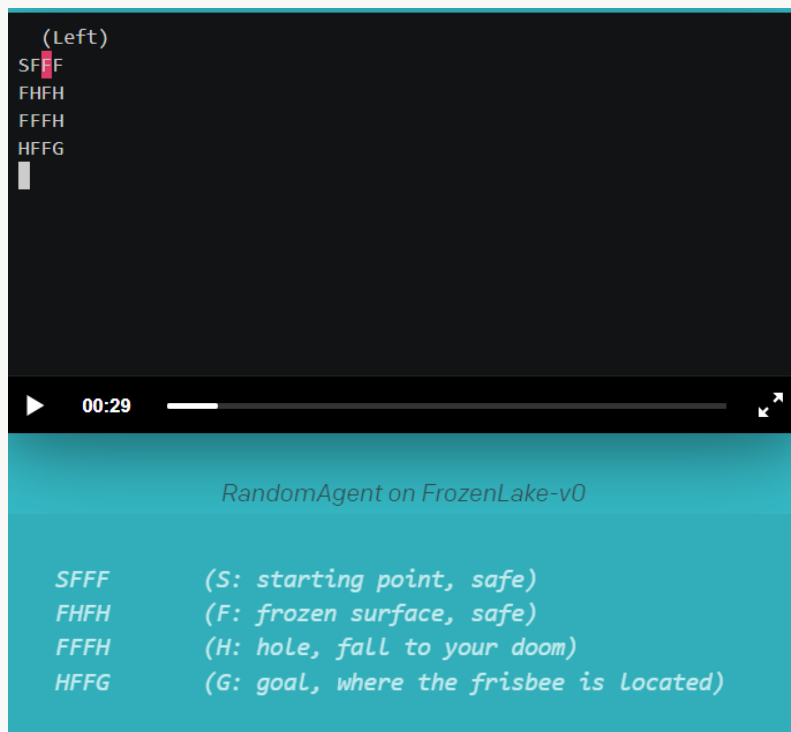
출력

47600

2. 강화학습 실습

1) Frozen Lake를 이용한 강화학습 실습

Frozen Lake란?



에이전트는 그리드 상에서 캐릭터의 움직임을 제어할 수 있다.

그리드의 F(Frozen surface) 타일은 걸을 수 있고 H(Hole) 타일은 에이전트가 물에 빠지게 되면서 게임이 종료된다.

에이전트의 이동 방향은 불확실하며 선택한 방향에 부분적으로만 의존.

에이전트는 목표 타일G(Goal)까지 걸어갈 수 있는 경로를 찾은 것에 대해 보상 획득.

2. 강화학습 실습

1) Frozen Lake를 이용한 강화학습 실습

```
import gym
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
```

프로즌 레이크 환경생성

```
env = gym.make('FrozenLake-v0', is_slippery=False)
```

```
print('환경 : ', env)
```

```
print('액션의 수 : ', env.action_space)
```

```
print('상태의 수 : ', env.observation_space) <실행결과>
```

```
print('시작 위치 : ', env.reset())
```

```
print('반환값 : ', env.step(1))
```

```
env.render()
```

환경 : <TimeLimit<FrozenLakeEnv<FrozenLake-v0>>>

액션의 수 : Discrete(4)

상태의 수 : Discrete(16)

시작 위치 : 0

반환값 : (4, 0.0, False, {'prob': 1.0})
(Down)

SFFF

■ HFH

FFFH

HFFG

2. 강화학습 실습

1) Frozen Lake를 이용한 강화학습 실습

난수로 학습을 진행하는 경우

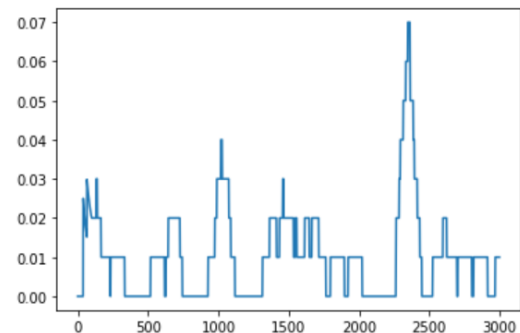
```
##### 3-2-2.ipynb #####
reward_list = []
reward_100_average = []

for epi in range(num_episode):
    if (is_video_save):
        txt = 'Episode : '+str(epi)
        for _ in range(3):
            out.write(np.uint8(draw_txt(txt)))
    d = False
    total_reward = 0
    s = env.reset()
    # 게임이 종료될 때까지 반복
    while not d:
        if (is_video_save):
            out.write(np.uint8(draw_state(s, q_table)))
        n_s, r, d, _ = env.step(env.action_space.sample())
        # env.action_space.sample()은 임의의 행동(0~3 사이의 값)을 반환.
        total_reward = total_reward+r
        s = n_s
    reward_list.append(total_reward)
    # 진행상태확인.
    # print(epi,np.average(reward_list[-100:]))
    reward_100_average.append(np.average(reward_list[-100:]))

print('{}번 시도 중 {}번 성공!'.format(num_episode, np.sum(reward_list)))
print('성공률은 {}% 입니다.'.format(round(np.sum(reward_list)/num_episode*100, 2)))
plt.plot(reward_100_average)
```

<실행결과>

3000번 시도 중 32.0번 성공!
성공률은 1.07% 입니다.
[<matplotlib.lines.Line2D at 0x7f6a0840f3d0>]



2. 강화학습 실습

1) Frozen Lake를 이용한 강화학습 실습

학습이 진행될 수록 랜덤하게 움직이는 비율을 낮추어 실행

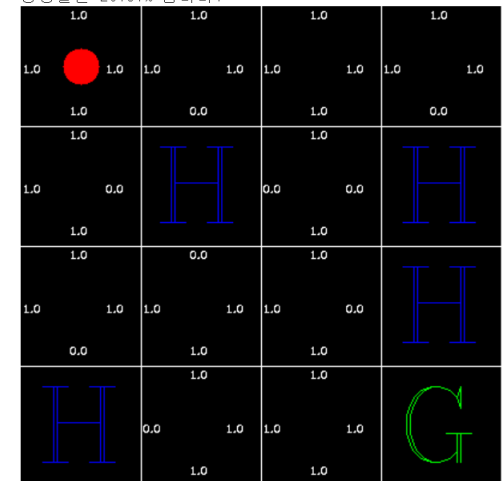
```
##### 3-2-3.ipynb #####
epsilon_max = 0.9
epsilon_min = 0.1
epsilon_count = 2000
epsilon = epsilon_max
epsilon_decay = epsilon_min/epsilon_max
epsilon_decay = epsilon_decay**(1./float(epsilon_count))
for epi in range(num_episode):
    d = False
    total_reward = 0
    s = env.reset()
    while not d:
        # 랜덤함수를 사용하여 일정비율로는 랜덤하게 움직이고,
        # 나머지 비율로는 이전처럼 큰 Q값을 따라 다녀보자
        if (np.random.rand()) < epsilon:
            action = env.action_space.sample()
        else:
            action = np.argmax(np.random.random((4,))*(q_table[s] == q_table[s].max()))
        n_s, r, d, _ = env.step(action)
        total_reward = total_reward+r
        q_table[s][action] = r+np.max(q_table[n_s])
        s = n_s

    reward_list.append(total_reward)

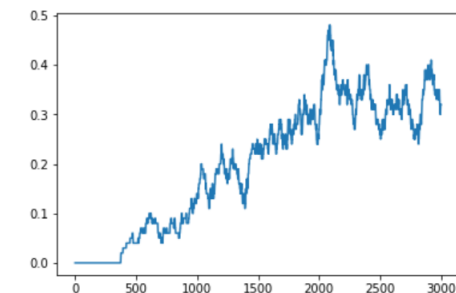
    if (epsilon > epsilon_min):
        epsilon = epsilon*epsilon_decay
    reward_100_average.append(np.average(reward_list[-100:]))
```

<실행결과>

3000번 시도 중 620.0번 성공!
성공률은 20.67% 입니다.



[<matplotlib.lines.Line2D at 0x7ff91ad3be50>]



2. 강화학습 실습

1) Frozen Lake를 이용한 강화학습 실습

Q값을 갱신할 때 할인율 적용

```
##### 3-2-4.ipynb #####
```

```
# 할인율 : 0.9
```

```
discount_rate = 0.9
```

```
epsilon = epsilon_max
```

```
epsilon_decay = epsilon_min/epsilon_max
```

```
epsilon_decay = epsilon_decay**(1./float(epsilon_count))
```

```
for epi in range(num_episode):    d = False
```

```
    total_reward = 0
```

```
    s = env.reset()
```

```
    while not d:
```

```
        if (is_video_save):
```

```
            out.write(np.uint8(draw_state(s, q_table)))
```

```
        if (np.random.rand()) < epsilon:
```

```
            action = env.action_space.sample()
```

```
        else:
```

```
            action = np.argmax(np.random.random((4,))*(q_table[s] == q_table[s].max()))
```

```
        n_s, r, d, _ = env.step(action)
```

```
        total_reward = total_reward+r
```

```
        # Q값을 업데이트 해줄때 할인율을 곱해서 해주자.
```

```
        q_table[s][action] = r+discount_rate*np.max(q_table[n_s])
```

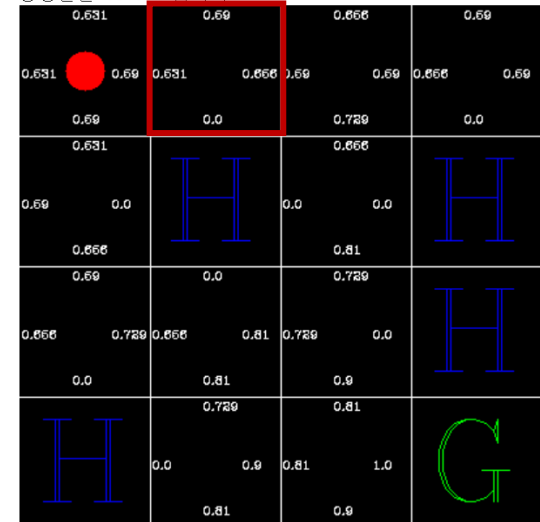
```
        s = n_s
```

```
    reward_list.append(total_reward)
```

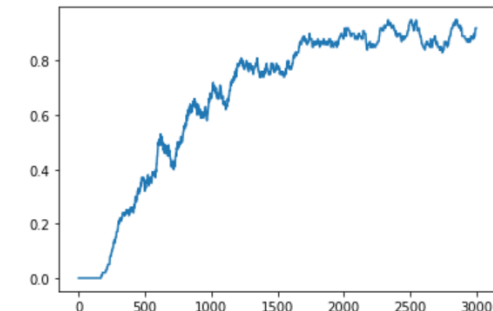
<실행결과>

3000번 시도 중 2050.0번 성공!

성공률은 68.33% 입니다.



[<matplotlib.lines.Line2D at 0x7fa7f0d8c450>]



2. 강화학습 실습

1) Frozen Lake를 이용한 강화학습 실습

외부조건 추가(slippery)

```
##### 3-2-5.ipynb #####  
# 프로즌 레이크 환경생성
```

```
env = gym.make('FrozenLake-v0', is_slippery=True)
```

```
q_table = np.zeros((16, 4))
```

```
discount_rate = 0.9
```

```
epsilon = epsilon_max
```

```
epsilon_decay = epsilon_min/epsilon_max
```

```
epsilon_decay = epsilon_decay**(1./float(epsilon_count))
```

```
for epi in range(num_episode):    d = False
```

```
    total_reward = 0
```

```
    s = env.reset()
```

```
    while not d:
```

```
        if (is_video_save):
```

```
            out.write(np.uint8(draw_state(s, q_table)))
```

```
        if (np.random.rand()) < epsilon:
```

```
            action = env.action_space.sample()
```

```
        else:
```

```
            action = np.argmax(np.random.random((4,))*(q_table[s] == q_table[s].max()))
```

```
        n_s, r, d, _ = env.step(action)
```

```
        total_reward = total_reward+r
```

```
    # Q값을 업데이트 해줄때 할인율을 곱해서 해주자.
```

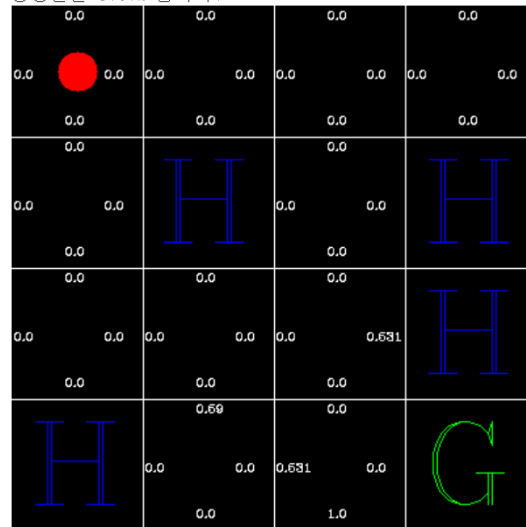
```
    q_table[s][action] = r+discount_rate*np.max(q_table[n_s])
```

```
    s = n_s
```

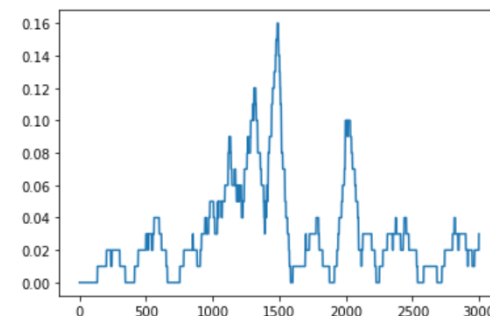
```
    reward_list.append(total_reward)
```

<실행결과>

3000번 시도 중 92.0번 성공!
성공률은 3.07% 입니다.



[<matplotlib.lines.Line2D at 0x7fa350aef3d0>]



2. 강화학습 실습

1) Frozen Lake를 이용한 강화학습 실습

학습률 적용

```
##### 3-2-6.ipynb #####
# 학습률 적용
# 에피소드 수를 10배증가 .
num_episode = 30000

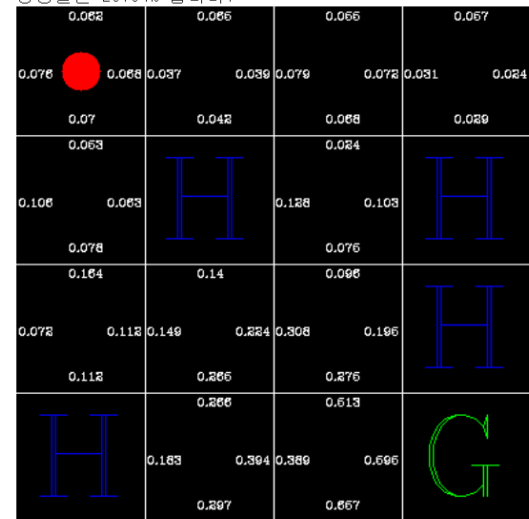
# 학습률 1%로 설정
learning_rate = 0.01
discount_rate = 0.9
for epi in range(num_episode):
    total_reward = 0
    s = env.reset()
    while not d:
        if (is_video_save and epi%video_div == 0):
            out.write(np.uint8(draw_state(s, q_table)))
        if (np.random.rand()) < epsilon:
            action = env.action_space.sample()
        else:
            action = np.argmax(np.random.random((4,))*(q_table[s] == q_table[s].max()))
        n_s, r, d, _ = env.step(action)
        total_reward = total_reward+r
        # 멘토의 의견은 1% 내가 가지고 있는 값 99%
        q_table[s][action] = (1-learning_rate)*q_table[s][action] + \
            learning_rate*(r+discount_rate*np.max(q_table[n_s]))
        s = n_s

    reward_list.append(total_reward)

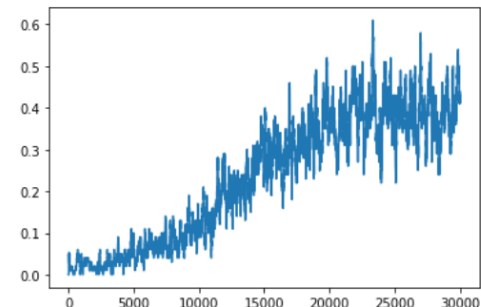
if (epsilon > epsilon_min):
    epsilon = epsilon*epsilon_decay
```

<실행결과>

30000번 시도 중 7001.0번 성공!
성공률은 23.34% 입니다.



[<matplotlib.lines.Line2D at 0x7fd63308d0d0>]



2. 강화학습 실습

1) Frozen Lake를 이용한 강화학습 실습

정책 적용

```
##### 3-2-7.ipynb #####  
# 정책 적용  
# 초기에 각 액션의 확률은 1/4로 맞춰준다.
```

```
p_table = np.zeros((16, 4))  
for i in range(16):  
    for j in range(4):  
        p_table[i, j] = 0.25
```

```
avi_file_name = '3-2-7.avi'
```

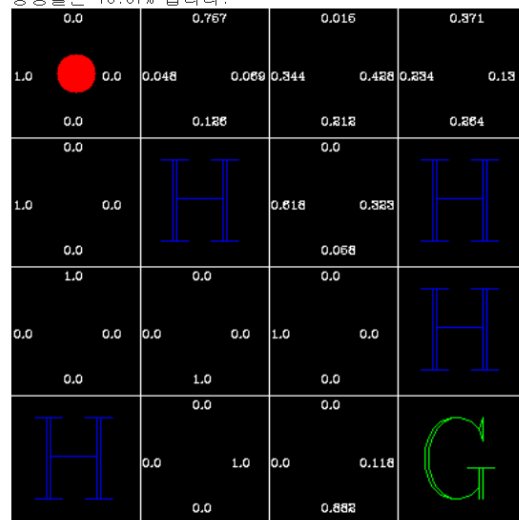
```
# 칭찬할 때 쓰이는 비율  
learning_rate = 0.01  
discount_rate = 0.9  
for epi in range(num_episode):
```

```
    .....  
    while not d:  
        if (is_video_save and epi%video_div == 0):  
            out.write(np.uint8(draw_state(s, p_table)))  
        action = np.random.choice(range(4), p=p_table[s])  
        n_s, r, d, _ = env.step(action)  
        total_reward = total_reward+r  
        i = (s, action, r, n_s, d)  
        memory.append(i)  
        s = n_s
```

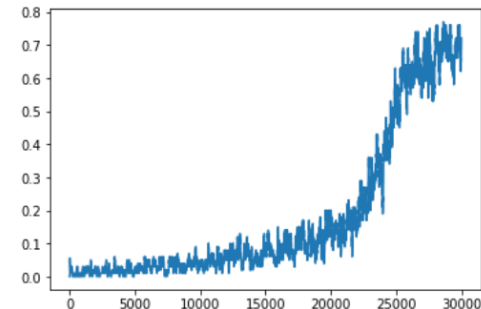
```
    r_sum = 0  
    for _s, _a, _r, _n_s, _d in memory[::-1]:  
        r_sum = _r+discount_rate*r_sum  
        log_p_table = np.log(p_table[_s])  
        log_p_table[_a] = log_p_table[_a]+learning_rate*r_sum/(p_table[_s, _a]+0.01)
```

<실행결과>

30000번 시도 중 5571.0번 성공!
성공률은 18.57% 입니다.



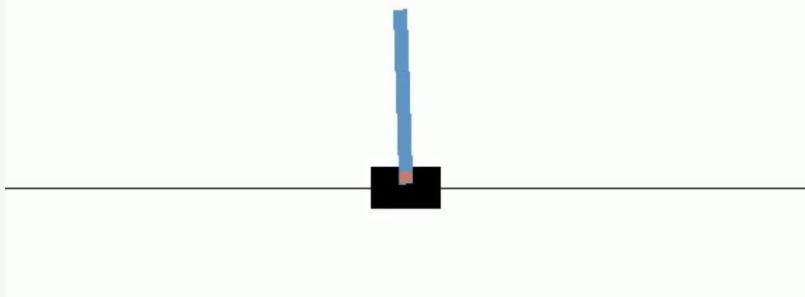
[<matplotlib.lines.Line2D at 0x7f3b17d63850>]



2. 강화학습 실습

2) Cart pole을 이용한 강화학습 실습

Cart pole이란?



풀은 마찰이 없는 트랙을 따라 움직이는 카트에 작동되지 않는 조인트에 의해 부착됩니다. 시스템은 카트에 +1 또는 -1의 힘을 가하여 제어됩니다. 진자는 똑바로 시작하고 목표는 넘어지는 것을 방지하는 것입니다. 기둥이 수직으로 유지되는 모든 타임스텝에 대해 +1의 보상이 제공됩니다. 극이 수직에서 15도 이상 떨어지거나 카트가 중심에서 2.4단위 이상 이동하면 에피소드가 종료됩니다.

2. 강화학습 실습

2) Cart pole을 이용한 강화학습 실습

Colab에서 Render를 그림파일로 해주기 위한 설정

```
!apt update
```

```
!apt-get install python-opengl -y
```

```
!apt install xvfb -y
```

```
!pip install pyvirtualdisplay
```

```
!pip install piglet
```

<실행결과>

시각화를 위한 라이브러리
설치

```
Ign:1 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu1804/x86_64 InRelease
Get:2 http://security.ubuntu.com/ubuntu bionic-security InRelease [88.7 kB]
Get:3 http://ppa.launchpad.net/c2d4u.team/c2d4u4.0+/ubuntu bionic InRelease [15.9 kB]
Get:4 https://cloud.r-project.org/bin/linux/ubuntu bionic-cran40/ InRelease [3,626 B]
Hit:5 http://archive.ubuntu.com/ubuntu bionic InRelease
Ign:6 https://developer.download.nvidia.com/compute/machine-learning/repos/ubuntu1804/x86_64 InRelease
Hit:7 https://developer.download.nvidia.com/compute/cuda/repos/ubuntu1804/x86_64 Release
Hit:8 https://developer.download.nvidia.com/compute/machine-learning/repos/ubuntu1804/x86_64 Release
Get:9 http://archive.ubuntu.com/ubuntu bionic-updates InRelease [88.7 kB]
Hit:10 http://ppa.launchpad.net/cran/libgit2/ubuntu bionic InRelease
Get:11 http://archive.ubuntu.com/ubuntu bionic-backports InRelease [74.6 kB]
Get:12 http://ppa.launchpad.net/deadsnakes/ppa/ubuntu bionic InRelease [15.9 kB]
Get:13 https://cloud.r-project.org/bin/linux/ubuntu bionic-cran40/ Packages [73.9 kB]
Hit:14 http://ppa.launchpad.net/graphics-drivers/ppa/ubuntu bionic InRelease
Get:17 http://ppa.launchpad.net/c2d4u.team/c2d4u4.0+/ubuntu bionic/main amd64 Packages [933 kB]
Get:18 http://security.ubuntu.com/ubuntu bionic-security/main amd64 Packages [2,461 kB]
Get:19 http://archive.ubuntu.com/ubuntu bionic-updates/main amd64 Packages [2,898 kB]
Get:20 http://security.ubuntu.com/ubuntu bionic-security/restricted amd64 Packages [691 kB]
Get:21 http://security.ubuntu.com/ubuntu bionic-security/universe amd64 Packages [1,452 kB]
Get:22 http://ppa.launchpad.net/c2d4u.team/c2d4u4.0+/ubuntu bionic/main amd64 Packages [933 kB]
Get:23 http://archive.ubuntu.com/ubuntu bionic-updates/universe amd64 Packages [2,230 kB]
Get:24 http://archive.ubuntu.com/ubuntu bionic-backports/main amd64 Packages [11.6 kB]
Get:25 http://archive.ubuntu.com/ubuntu bionic-backports/universe amd64 Packages [12.6 kB]
Get:26 http://ppa.launchpad.net/deadsnakes/ppa/ubuntu bionic/main amd64 Packages [45.9 kB]
Fetched 12.9 MB in 4s (3,013 kB/s)
Reading package lists... Done
Building dependency tree
Reading state information... Done
76 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree
Reading state information... Done
Suggested packages:
  libgle3
The following NEW packages will be installed:
  python-opengl
0 upgraded, 1 newly installed, 0 to remove and 76 not upgraded.
```

2. 강화학습 실습

2) Cart pole을 이용한 강화학습 실습

3-3-1.ipynb

```
import tensorflow as tf
import gym
from IPython import display
import cv2
from pyvirtualdisplay import Display
from IPython import display
import matplotlib.pyplot as plt
from collections import deque
import numpy as np
import random
from google.colab.patches import cv2_imshow
%matplotlib inline
Display().start()

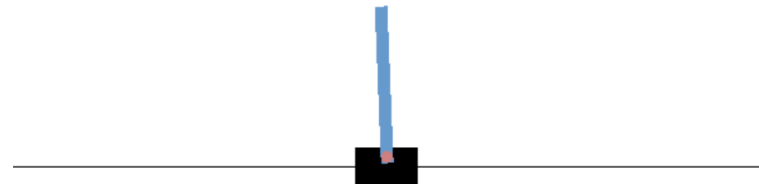
# 카트폴 환경을 만듭니다.
env = gym.make('CartPole-v1')
print('환경 : ', env)
print('행동할수 있는 액션의 수 : ', env.action_space)
print('이동할 수 있는 총 상태의 수 : ', env.observation_space)
print('시작 위치: ', env.reset())
print('결과값 ', env.step(0))
```

```
# 이미지로 렌더링 하고 이미지를 CV2로 보여주겠다.
cv2_imshow(env.render('rgb_array'))
```

<실행결과>

```
환경 : <TimeLimit<CartPoleEnv<CartPole-v1>>>
행동할수 있는 액션의 수 : Discrete(2)
이동할 수 있는 총 상태의 수 : Box(-3.4028234663852886e+38, 3.4028234663852886e+38, (4,), float32)
시작 위치: [-0.013129  0.04966597 -0.03982065 -0.00658703]
결과값 (array([-0.01213568, -0.14486295, -0.03995239,  0.27327085]), 1.0, False, {})
```

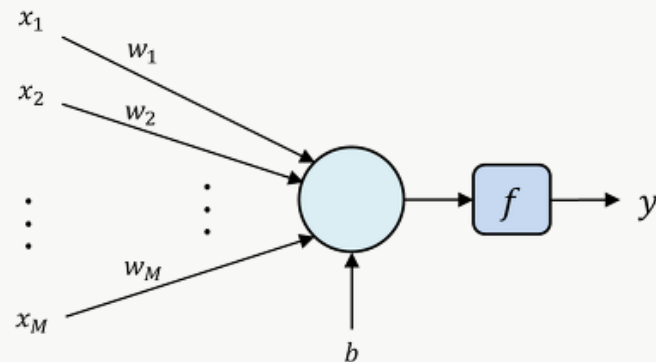
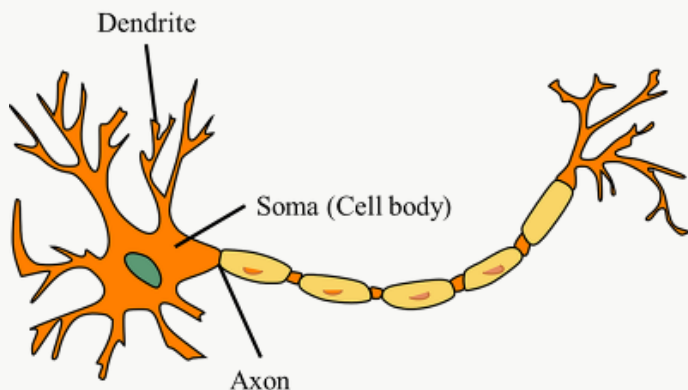
[위치, 위치가 변하는 비율, 막대각도, 각도가 변하는 비율]



2. 강화학습 실습

2) Cart pole을 이용한 강화학습 실습

신경망 학습이란?



Dendrite: 전기적 신호를 통해 입력 신호를 받는 기능을 수행한다. Artificial neuron에서는 벡터의 형태로 입력된 데이터 $x=[x_1x_2...x_M]^T$ 를 전달받는 역할을 한다.

Soma: dendrite를 통해 전달받은 입력을 합산하는 기능을 수행한다. Artificial neuron에서는 각각의 dendrite의 입력에 weight라고 하는 $w=[w_1w_2...x_M]^T$ 를 곱하여 합산한다. Artificial neuron에서 soma의 output $s(x;\theta)$ 는 $s(x;\theta)=\sum_{j=1}^M w_j x_j + b$ 로 정의되며, linear function의 형태를 갖는다. 이때 b 는 bias라고 하는 linear function의 상수항이며, θ 는 artificial neuron을 구성하는 parameter인 w 와 b 를 의미한다.

Axon: soma에서 계산된 값을 출력한다. Artificial neuron에서는 soma에서 출력된 $s(x;\theta)$ 를 activation function f 에 입력하여 계산된 output y 를 전달한다. 이때 activation function은 $s(x;\theta)$ 를 바탕으로 어떠한 결정을 내리는 기능을 수행한다. Activation function으로는 unit step function, sigmoid function, cross entropy 등 다양한 형태의 function을 이용될 수 있다.

2. 강화학습 실습

2) Cart pole을 이용한 강화학습 실습

신경망 모델로 가치함수 학습

```
##### 3-3-2.ipynb #####
```

```
state_num = (4,)
action_num = 2
hidden_state = 128
learning_rate = 0.001
```

```
model=tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(hidden_state,
                                input_shape=state_num,
                                activation='relu'))
model.add(tf.keras.layers.Dense(action_num))
model.compile(loss='mse',optimizer=tf.keras.optimizers.Adam(learning_rate))
```

```
model.summary()
```

<실행결과>

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	640
dense_1 (Dense)	(None, 2)	258

Total params: 898
Trainable params: 898
Non-trainable params: 0

2. 강화학습 실습

2) Cart pole을 이용한 강화학습 실습

신경망 모델로 가치함수 학습

```
##### 3-3-2.ipynb #####
for epi in range(num_episode):
    if (is_video_save):
        txt = 'Episode : '+str(epi)
        for _ in range(3):
            out.write(np.uint8(draw_txt(txt)))
    d = False
    total_reward = 0
    s = env.reset()
    s = np.reshape(s, [1, 4])

    while not d:
        # 현재 상태의 Q값을 구한다.
        q_val = model.predict(s)
        if (is_video_save):
            out.write(np.uint8(draw_state(env.render('rgb_array'), q_val[0])))

        # 특정값(입실론)을 기준으로 랜덤하게 움직인다.
        if (np.random.rand()) < epsilon:
            action = env.action_space.sample()
        else:
            action = np.argmax(q_val[0])
        n_s, r, d, _ = env.step(action)
        total_reward = total_reward+r
        n_s = np.reshape(n_s, [1, 4])

        #  $Q(s,a) = r + \text{discount\_rate} * \max(Q(s'))$ 
        # 위 수식을 그대로 구현해 준다.
        target_q_val = r+discount_rate*np.max(model.predict(n_s)[0])
```

```
#####
    if d:
        q_val[0][action] = r
    else:
        q_val[0][action] = target_q_val

    # 신경망 학습
    model.train_on_batch(np.array(s), np.array(q_val))
    s = n_s

    if (total_reward >= 300):
        d = True

    if (epsilon > epsilon_min):
        epsilon = epsilon*epsilon_decay

    reward_list.append(total_reward)

    if (is_video_save):
        out.write(np.uint8(draw_state(env.render('rgb_array'), q_val[0])))
        txt = 'Result : '+str(total_reward)
        for _ in range(3):
            out.write(np.uint8(draw_txt(txt)))
    print('현재 에피소드 : {}, 현재 점수 : {}, 현재 입실론 : {}'.format(epi,
        total_reward,
        round(epsilon*100, 2)))
```

2. 강화학습 실습

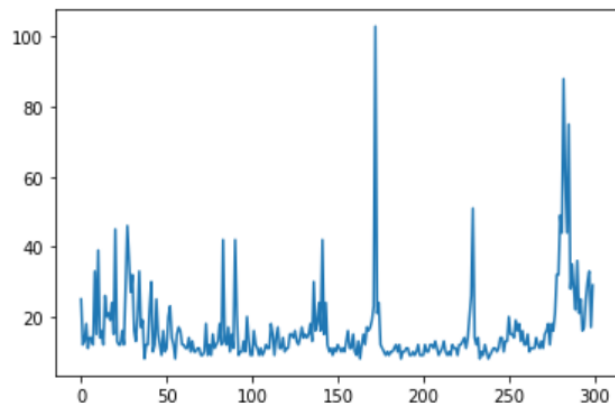
2) Cart pole을 이용한 강화학습 실습

신경망 모델로 가치함수 학습

<실행결과>

현재 에피소드 : 0, 현재 점수 : 25.0, 현재 입실론 : 89.21
현재 에피소드 : 1, 현재 점수 : 12.0, 현재 입실론 : 88.43
현재 에피소드 : 2, 현재 점수 : 13.0, 현재 입실론 : 87.66
현재 에피소드 : 3, 현재 점수 : 18.0, 현재 입실론 : 86.89
현재 에피소드 : 4, 현재 점수 : 11.0, 현재 입실론 : 86.13
현재 에피소드 : 5, 현재 점수 : 14.0, 현재 입실론 : 85.38

현재 에피소드 : 292, 현재 점수 : 25.0, 현재 입실론 : 10.0
현재 에피소드 : 293, 현재 점수 : 16.0, 현재 입실론 : 10.0
현재 에피소드 : 294, 현재 점수 : 17.0, 현재 입실론 : 10.0
현재 에피소드 : 295, 현재 점수 : 25.0, 현재 입실론 : 10.0
현재 에피소드 : 296, 현재 점수 : 30.0, 현재 입실론 : 10.0
현재 에피소드 : 297, 현재 점수 : 33.0, 현재 입실론 : 10.0
현재 에피소드 : 298, 현재 점수 : 17.0, 현재 입실론 : 10.0
현재 에피소드 : 299, 현재 점수 : 29.0, 현재 입실론 : 10.0



2. 강화학습 실습

2) Cart pole을 이용한 강화학습 실습

리플레이 버퍼의 적용

```
##### 3-3-3.ipynb #####
# 상태와 액션 그리고 리워드와 다음 상태를 저장할 리스트를 만든다
memory_size = 2000
memory = deque(maxlen=memory_size)
# 기록을 저장해 두었다가 한번에 학습할 량을 정한다
batch_size = 64

if (is_video_save):
    fcc = cv2.VideoWriter_fourcc(*'DIVX')
    out = cv2.VideoWriter(avi_file_name, fcc, fps, (600, 400))

reward_list = []

for epi in range(num_episode):
    if (is_video_save):
        txt = 'Episode : '+str(epi)
        for _ in range(3):
            out.write(np.uint8(draw_txt(txt)))
    d = False
    total_reward = 0
    s = env.reset()
    s = np.reshape(s, [1, 4])
```

```
#####
while not d:
    q_val = model.predict(s)
    if (is_video_save):
        out.write(np.uint8(draw_state(env.render('rgb_array'), q_val[0])))
    if (np.random.rand()) < epsilon:
        action = env.action_space.sample()
    else:
        action = np.argmax(q_val[0])
    n_s, r, d, _ = env.step(action)
    n_s = np.reshape(n_s, [1, 4])
    # 상태/액션/보상/다음상태 를 저장한다.
    i = (s, action, r/100., n_s, d)
    memory.append(i)
    s = n_s
    total_reward = total_reward+r

    if (total_reward >= 300):
        d = True
# 버퍼에 기록이 일정 이상 쌓이면 학습을 시작한다.
if len(memory) >= batch_size*2:
    # 버퍼의 기록 중 batch size 만큼 가져와서 학습을 진행한다.
    sample = random.sample(memory, batch_size)
    state_batch = []
    q_val_batch = []
    for _s, _a, _r, _n_s, _d in sample:
        q_val = model.predict(_s)
        target_q_val = _r+discount_rate*np.max(model.predict(_n_s)[0])
        if _d:
            q_val[0][_a] = _r
        else:
            q_val[0][_a] = target_q_val
        state_batch.append(_s[0])
        q_val_batch.append(q_val[0])
    model.train_on_batch(np.array(state_batch), np.array(q_val_batch))
```

2. 강화학습 실습

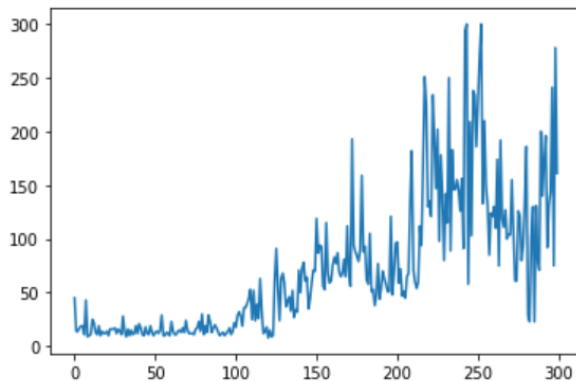
2) Cart pole을 이용한 강화학습 실습

리플레이 버퍼의 적용

<실행결과>

현재 에피소드 : 0, 현재 점수 : 45.0, 현재 입실론 : 89.21
현재 에피소드 : 1, 현재 점수 : 14.0, 현재 입실론 : 88.43
현재 에피소드 : 2, 현재 점수 : 14.0, 현재 입실론 : 87.66
현재 에피소드 : 3, 현재 점수 : 17.0, 현재 입실론 : 86.89
현재 에피소드 : 4, 현재 점수 : 19.0, 현재 입실론 : 86.13
현재 에피소드 : 5, 현재 점수 : 19.0, 현재 입실론 : 85.38
현재 에피소드 : 6, 현재 점수 : 11.0, 현재 입실론 : 84.63
현재 에피소드 : 7, 현재 점수 : 43.0, 현재 입실론 : 83.89
현재 에피소드 : 8, 현재 점수 : 9.0, 현재 입실론 : 83.16
현재 에피소드 : 9, 현재 점수 : 10.0, 현재 입실론 : 82.43
현재 에피소드 : 10, 현재 점수 : 11.0, 현재 입실론 : 81.71

현재 에피소드 : 293, 현재 점수 : 92.0, 현재 입실론 : 10.0
현재 에피소드 : 294, 현재 점수 : 131.0, 현재 입실론 : 10.0
현재 에피소드 : 295, 현재 점수 : 144.0, 현재 입실론 : 10.0
현재 에피소드 : 296, 현재 점수 : 241.0, 현재 입실론 : 10.0
현재 에피소드 : 297, 현재 점수 : 75.0, 현재 입실론 : 10.0
현재 에피소드 : 298, 현재 점수 : 278.0, 현재 입실론 : 10.0
현재 에피소드 : 299, 현재 점수 : 161.0, 현재 입실론 : 10.0



2. 강화학습 실습

2) Cart pole을 이용한 강화학습 실습

타겟 모델

```
##### 3-3-4ipynb #####
```

```
# 모델을 새로 정의하자
```

```
model = tf.keras.models.Sequential()
```

```
model.add(tf.keras.layers.Dense(hidden_state,
                                input_shape=state_num,
                                activation='relu'))
```

```
model.add(tf.keras.layers.Dense(action_num))
```

```
model.compile(loss='mse',
```

```
optimizer=tf.keras.optimizers.Adam(learning_rate))
```

```
# 타겟 모델을 정의해 준다.
```

```
target_model = tf.keras.models.clone_model(model)
```

```
# 타겟을 몇번 에피소드마다 업데이트 할 지 정한다.
```

```
target_update_count = 30
```

```
.....
```

```
while not d:
```

```
    q_val = model.predict(s)
```

```
    if (is_video_save):
```

```
        out.write(np.uint8(draw_state(env.render('rgb_array'), q_val[0])))
```

```
    if (np.random.rand()) < epsilon:
```

```
        action = env.action_space.sample()
```

```
    else:
```

```
        action = np.argmax(q_val[0])
```

```
    n_s, r, d, _ = env.step(action)
```

```
    n_s = np.reshape(n_s, [1, 4])
```

```
    i = (s, action, r/100., n_s, d)
```

```
    memory.append(i)
```

```
    s = n_s
```

```
    total_reward = total_reward+r
```

```
    if (total_reward >= 300):
```

```
        d = True
```

```
#####
```

```
if len(memory) >= batch_size*2:
```

```
    sample = random.sample(memory, batch_size)
```

```
    state_batch = []
```

```
    q_val_batch = []
```

```
    for _s, _a, _r, _n_s, _d in sample:
```

```
        q_val = model.predict(_s)
```

```
        # Q값을 업데이트할때 model을 사용하지 않고, target_model을 사용한다.
```

```
        target_q_val = _r+discount_rate*np.max(target_model.predict(_n_s)[0])
```

```
        if _d:
```

```
            q_val[0][_a] = _r
```

```
        else:
```

```
            q_val[0][_a] = target_q_val
```

```
    state_batch.append(_s[0])
```

```
    q_val_batch.append(q_val[0])
```

```
    model.train_on_batch(np.array(state_batch), np.array(q_val_batch))
```

```
    train_count = train_count+1
```

2. 강화학습 실습

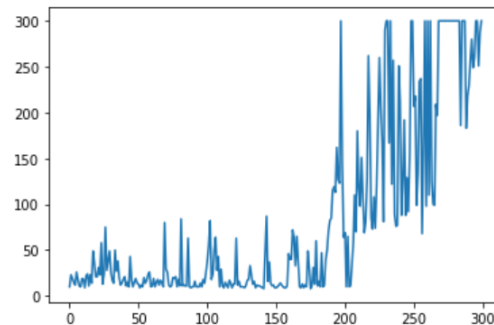
2) Cart pole을 이용한 강화학습 실습

타겟 모델

<실행결과>

```
현재 에피소드 : 0, 현재 점수 : 10.0, 현재 임실론 : 89.21
현재 에피소드 : 1, 현재 점수 : 23.0, 현재 임실론 : 88.43
현재 에피소드 : 2, 현재 점수 : 19.0, 현재 임실론 : 87.66
현재 에피소드 : 3, 현재 점수 : 15.0, 현재 임실론 : 86.89
현재 에피소드 : 4, 현재 점수 : 12.0, 현재 임실론 : 86.13
현재 에피소드 : 5, 현재 점수 : 26.0, 현재 임실론 : 85.38
현재 에피소드 : 6, 현재 점수 : 18.0, 현재 임실론 : 84.63
현재 에피소드 : 7, 현재 점수 : 11.0, 현재 임실론 : 83.89
현재 에피소드 : 8, 현재 점수 : 10.0, 현재 임실론 : 83.16
현재 에피소드 : 9, 현재 점수 : 19.0, 현재 임실론 : 82.43
현재 에피소드 : 10, 현재 점수 : 18.0, 현재 임실론 : 81.71
현재 에피소드 : 11, 현재 점수 : 9.0, 현재 임실론 : 80.99
현재 에피소드 : 12, 현재 점수 : 22.0, 현재 임실론 : 80.28
현재 에피소드 : 13, 현재 점수 : 24.0, 현재 임실론 : 79.58
현재 에피소드 : 14, 현재 점수 : 11.0, 현재 임실론 : 78.88
현재 에피소드 : 15, 현재 점수 : 23.0, 현재 임실론 : 78.19
현재 에피소드 : 16, 현재 점수 : 14.0, 현재 임실론 : 77.51
현재 에피소드 : 17, 현재 점수 : 49.0, 현재 임실론 : 76.83
현재 에피소드 : 18, 현재 점수 : 36.0, 현재 임실론 : 76.16
현재 에피소드 : 19, 현재 점수 : 21.0, 현재 임실론 : 75.49
현재 에피소드 : 20, 현재 점수 : 21.0, 현재 임실론 : 74.83
현재 에피소드 : 21, 현재 점수 : 31.0, 현재 임실론 : 74.18
현재 에피소드 : 22, 현재 점수 : 23.0, 현재 임실론 : 73.53
현재 에피소드 : 23, 현재 점수 : 58.0, 현재 임실론 : 72.88
현재 에피소드 : 24, 현재 점수 : 13.0, 현재 임실론 : 72.25
현재 에피소드 : 25, 현재 점수 : 24.0, 현재 임실론 : 71.61
현재 에피소드 : 26, 현재 점수 : 75.0, 현재 임실론 : 70.99
현재 에피소드 : 27, 현재 점수 : 28.0, 현재 임실론 : 70.37
현재 에피소드 : 28, 현재 점수 : 38.0, 현재 임실론 : 69.75
현재 에피소드 : 29, 현재 점수 : 49.0, 현재 임실론 : 69.14
현재 에피소드 : 30, 현재 점수 : 26.0, 현재 임실론 : 68.54
현재 에피소드 : 31, 현재 점수 : 17.0, 현재 임실론 : 67.94
현재 에피소드 : 32, 현재 점수 : 14.0, 현재 임실론 : 67.34
현재 에피소드 : 33, 현재 점수 : 50.0, 현재 임실론 : 66.75
현재 에피소드 : 34, 현재 점수 : 27.0, 현재 임실론 : 66.17
Target Updated
```

```
현재 에피소드 : 274, 현재 점수 : 300.0, 현재 임실론 : 10.0
Target Updated
현재 에피소드 : 275, 현재 점수 : 300.0, 현재 임실론 : 10.0
현재 에피소드 : 276, 현재 점수 : 300.0, 현재 임실론 : 10.0
현재 에피소드 : 277, 현재 점수 : 300.0, 현재 임실론 : 10.0
현재 에피소드 : 278, 현재 점수 : 300.0, 현재 임실론 : 10.0
현재 에피소드 : 279, 현재 점수 : 300.0, 현재 임실론 : 10.0
현재 에피소드 : 280, 현재 점수 : 300.0, 현재 임실론 : 10.0
현재 에피소드 : 281, 현재 점수 : 300.0, 현재 임실론 : 10.0
현재 에피소드 : 282, 현재 점수 : 300.0, 현재 임실론 : 10.0
현재 에피소드 : 283, 현재 점수 : 300.0, 현재 임실론 : 10.0
현재 에피소드 : 284, 현재 점수 : 186.0, 현재 임실론 : 10.0
현재 에피소드 : 285, 현재 점수 : 300.0, 현재 임실론 : 10.0
현재 에피소드 : 286, 현재 점수 : 300.0, 현재 임실론 : 10.0
현재 에피소드 : 287, 현재 점수 : 300.0, 현재 임실론 : 10.0
현재 에피소드 : 288, 현재 점수 : 183.0, 현재 임실론 : 10.0
현재 에피소드 : 289, 현재 점수 : 218.0, 현재 임실론 : 10.0
현재 에피소드 : 290, 현재 점수 : 229.0, 현재 임실론 : 10.0
현재 에피소드 : 291, 현재 점수 : 256.0, 현재 임실론 : 10.0
현재 에피소드 : 292, 현재 점수 : 280.0, 현재 임실론 : 10.0
현재 에피소드 : 293, 현재 점수 : 249.0, 현재 임실론 : 10.0
현재 에피소드 : 294, 현재 점수 : 260.0, 현재 임실론 : 10.0
현재 에피소드 : 295, 현재 점수 : 300.0, 현재 임실론 : 10.0
현재 에피소드 : 296, 현재 점수 : 300.0, 현재 임실론 : 10.0
현재 에피소드 : 297, 현재 점수 : 251.0, 현재 임실론 : 10.0
현재 에피소드 : 298, 현재 점수 : 287.0, 현재 임실론 : 10.0
현재 에피소드 : 299, 현재 점수 : 300.0, 현재 임실론 : 10.0
```



2. 강화학습 실습

2) Cart pole을 이용한 강화학습 실습

학습된 모델을 이용해 게임만 진행

<실행결과>



```
model.save('3-3-4')
```

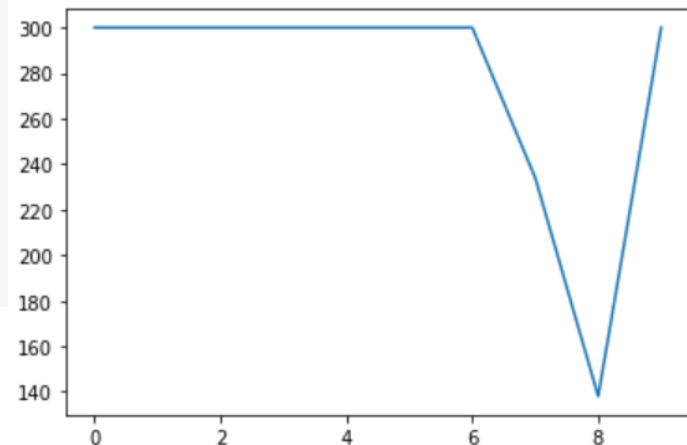
```
INFO:tensorflow:Assets written to: 3-3-4/assets
```

```
##모델 불러오기
model= tf.keras.models.load_model('/content/drive/MyDrive/Colab Notebooks/3-3-4')
model.summary()

# 파라미터들을 설정해 주자
# 에피소드를 10번만 실행
num_episode = 10
is_video_save = False
fps = 10.0
avi_file_name = '3-3-2_load_model.avi'

epsilon_max = 0.1
epsilon_min = 0.01
epsilon_count = 250
```

현재 에피소드	:	0	, 현재 점수	:	300.0	, 현재 입실론	:	9.91
현재 에피소드	:	1	, 현재 점수	:	300.0	, 현재 입실론	:	9.82
현재 에피소드	:	2	, 현재 점수	:	300.0	, 현재 입실론	:	9.73
현재 에피소드	:	3	, 현재 점수	:	300.0	, 현재 입실론	:	9.64
현재 에피소드	:	4	, 현재 점수	:	300.0	, 현재 입실론	:	9.55
현재 에피소드	:	5	, 현재 점수	:	300.0	, 현재 입실론	:	9.46
현재 에피소드	:	6	, 현재 점수	:	300.0	, 현재 입실론	:	9.38
현재 에피소드	:	7	, 현재 점수	:	234.0	, 현재 입실론	:	9.29
현재 에피소드	:	8	, 현재 점수	:	138.0	, 현재 입실론	:	9.2
현재 에피소드	:	9	, 현재 점수	:	300.0	, 현재 입실론	:	9.12



2. 강화학습 실습

2) Cart pole을 이용한 강화학습 실습

정책 신경망

```
##### 3-3-5.ipynb #####
# 상태(4가지값)를 입력 받으면,
# 각 액션(2가지)의 Q값을 돌려주는 신경망을 모델링하자
# 마지막 출력층은 확률값이 나와야 하므로 softmax를 사용하자
```

```
state_num = (4,)
action_num = 2
hidden_state = 128
learning_rate = 0.001
```

```
model = tf.keras.models.Sequential()
model.add(tf.keras.layers.Dense(hidden_state,
                                input_shape=state_num,
                                activation='relu'))
model.add(tf.keras.layers.Dense(action_num, activation='softmax'))
```

```
opt = tf.keras.optimizers.Adam(learning_rate)
model.summary()
```

relu	은닉 층으로 학습
sigmond	이진 분류 문제
softmax	클래스 분류 문제

```
#####
for epi in range(num_episode):
    if (is_video_save):
        txt = 'Episode : '+str(epi)
        for _ in range(3):
            out.write(np.uint8(draw_txt(txt)))
    d = False
    total_reward = 0
    s = env.reset()
    s = np.reshape(s, [1, 4])
    memory = []
    while not d:
        p = model.predict(s)[0]
        if (is_video_save):
            out.write(np.uint8(draw_state(env.render('rgb_array'), p)))
        action = np.random.choice(range(2), p=p)
        n_s, r, d, _ = env.step(action)
        n_s = np.reshape(n_s, [1, 4])
        i = (s, action, r/100., n_s, d)
        memory.append(i)
        s = n_s
        total_reward = total_reward+r
        if (total_reward >= 300):
            d = True
    r_sum = 0
    for _s, _a, _r, _n_s, _d in memory[::-1]:
        r_sum = _r+discount_rate*r_sum
        variable = model.trainable_variables
        with tf.GradientTape() as tape:
            p = model(_s)[0][_a]
            # -log(p)의 미분은 -1/p가 되어 이전 식과 같아진다.
            loss = -tf.math.log(p)*r_sum
        grad = tape.gradient(loss, variable)
        opt.apply_gradients(zip(grad, variable))

reward_list.append(total_reward)
```

reward_list.append(total_reward)

2. 강화학습 실습

2) Cart pole을 이용한 강화학습 실습

정책 신경망

<실행결과>

현재 에피소드	:	0,	현재 점수	:	17.0
현재 에피소드	:	1,	현재 점수	:	19.0
현재 에피소드	:	2,	현재 점수	:	20.0
현재 에피소드	:	3,	현재 점수	:	24.0
현재 에피소드	:	4,	현재 점수	:	15.0
현재 에피소드	:	5,	현재 점수	:	31.0
현재 에피소드	:	6,	현재 점수	:	25.0
현재 에피소드	:	7,	현재 점수	:	21.0
현재 에피소드	:	8,	현재 점수	:	19.0
현재 에피소드	:	9,	현재 점수	:	33.0
현재 에피소드	:	10,	현재 점수	:	21.0

현재 에피소드	:	291,	현재 점수	:	113.0
현재 에피소드	:	292,	현재 점수	:	224.0
현재 에피소드	:	293,	현재 점수	:	155.0
현재 에피소드	:	294,	현재 점수	:	242.0
현재 에피소드	:	295,	현재 점수	:	167.0
현재 에피소드	:	296,	현재 점수	:	251.0
현재 에피소드	:	297,	현재 점수	:	199.0
현재 에피소드	:	298,	현재 점수	:	300.0
현재 에피소드	:	299,	현재 점수	:	300.0

